



JevBench: Metamorphic Coherence Testing for Typed Probabilistic Decision Models

Chen Feng

ML Lab, Queen's University Belfast

✉ c.feng@qub.ac.uk

🌐 jevbench.github.io  github.com/JevBench/jevbench  pypi.org/project/jevbench

Abstract

Typed probabilistic decision models answer questions about a fixed input with probability distributions over declared answers: yes or no, one of several options, or a level on an ordered scale. Beyond the robustness expected of any language model, these classification-like outputs raise a further question: do their probabilities stay coherent when a question is reworded, negated or logically combined with others, offered a different set of options, or asked together with other questions? We introduce JevBench, a benchmark that measures this coherence by metamorphic testing, without gold labels. It defines 50 metamorphic relations, each a single law from probability or choice theory, in five dimensions: representation, batch independence, probability measure, logical order and choice-set conditioning. Every test transforms only the question side, never the input, and all tests are frozen in advance, so every model answers the same requests. The benchmark comprises 240 synthetic cases in 12 decision domains, with 12,000 tests (JevBench-240) and a reported subset of 1,200 tests (JevBench-mini) that closely reproduces the full-suite ranking (Spearman 0.978 across 17 models). We evaluate 21 openly released models. Coherence scores range from 56.5 to 83.9; batch independence holds for most models, while additivity fails broadly, above all for complements, where a question and its negation should sum to one. A reference model that ignores its input and answers uniformly scores 89.4, above every tested model, at chance accuracy: coherence is necessary for a trustworthy decision model but not sufficient, and we report it beside accuracy. Among decoder models, accuracy varies little (82.4–89.2%) while coherence varies widely (64.2–83.9), so the two measures separate models that accuracy alone does not. JevBench is released as an open-source Python package.

1 Introduction

A growing class of small models answers typed questions about a piece of text: is this message a refund request, which team should own this ticket, how urgent is this incident on a four-level scale. Each answer is a probability distribution over declared outcomes: a yes/no question (a *Noul*), a choice among named options (a *Choice*) or a level on an ordered scale (a *Score*). The models differ widely in how they are built (encoders that score one marker per option, fine-tuned decoders that read the logits of answer letters, pointer and scorer heads on decoder backbones, contrastive embeddings), but many of them now implement the same wire interface, which we call Jev: a request carries a fixed *state* and a batch of typed questions, and the response returns one distribution per question.¹

These outputs are classification-like, so beyond the robustness expected of any language model they raise a further question: are the probabilities themselves coherent? A question and its negation should sum to one; merging two options should merge their probabilities; a conjunction should never be more probable than either conjunct; removing an irrelevant option should not reorder the others; and the answer to a question should not depend on which other questions were asked with it. These laws matter in practice, because the probabilities are rarely read one at a time: a routing system compares the probability of each team, a moderation pipeline thresholds the probability of a violation, and a workflow negates a condition, merges two options or removes one that does not apply. A model that violates them can be accurate on every question in isolation and still produce inconsistent decisions once its answers are combined.

¹The interface is a single HTTP endpoint, `POST /v1/systemone`. JevBench is not affiliated with any provider of the interface; “Jev” only names it.

Accuracy benchmarks do not measure this. They need gold labels for each domain, and they score each answer separately. Coherence, by contrast, can be tested without labels: the laws of probability and choice say how the answers to *related* questions must relate, whatever the correct answers are. This is the setting of metamorphic testing [3, 23]: transform an input in a way whose effect on the output is known, and check that the output changes accordingly.

We introduce JevBench, a metamorphic coherence benchmark for typed probabilistic decision models. Its contributions are:

- **A formal definition** of a metamorphic relation for typed probabilistic outputs, with three forms of oracle (distributional equality, identity, inequality) and one violation measure for each, all in probability units, so that one tolerance is meaningful across relations (§3).
- **A taxonomy of 50 relations** in five dimensions, classified by the operation that links the two compared distributions: representation (REP), batch independence (BAT), probability measure (MEA), logical order (LOG) and choice-set conditioning (CHO). Each relation states exactly one law, and the relations are irredundant (§3.7, Appendix A).
- **A frozen benchmark** of 240 synthetic cases in 12 decision domains, generated fact-first so that every question has a rule-based gold annotation (an explicit unknown when the facts it needs are absent), with every test and every request fixed in advance: JevBench-240, with 12,000 tests, and JevBench-mini, a fixed subset of 1,200 tests that we report (§4).
- **An evaluation of 21 openly released models** from seven architecture families, served with their developers’ own code, together with reference models and the accuracy of every model’s unmodified answers (§5).
- **An open-source package**, `pip install jevbench`, that runs the benchmark against any server implementing the interface or any Python function, with a request cache and replicas.

The evaluation shows that coherence and accuracy measure different things. Scores range from 56.5 to 83.9. Batch independence holds for 19 of the 21 models; the two exceptions follow from how each model reads a batch (§5.4). Probability-measure coherence is the weakest dimension for every model, and its complement group, where a question and its negation must sum to one, is the weakest group: no model scores above 32.5 on it, and a templated negation (for example “Is it false that the answer to the following question is yes?”) sums to one with its source question in 7% of tests on average. A reference model that answers every question uniformly, and so ignores its input, scores 89.4, above every real model, at chance accuracy (39.8%). Among the 14 decoder models, accuracy lies between 82.4% and 89.2%, while coherence ranges from 64.2 to 83.9. Coherence is therefore no substitute for accuracy, and accuracy no substitute for coherence; we report the two side by side.

2 Related work

Metamorphic and behavioural testing. Metamorphic testing checks a program against relations between the outputs of related inputs when no oracle for a single output exists [3, 23]. It has been applied to machine learning classifiers, where relations such as permuting the training data or the attributes must leave predictions unchanged [27]. CheckList [21] brought behavioural testing to NLP models with invariance and directional tests built from templates and perturbations. JevBench follows this line, but its relations are laws of probability and choice over distributions, measured by how far each law fails rather than by whether a label flips, and its test set is frozen so that every model answers exactly the same requests.

Consistency of language models. Pretrained language models give inconsistent answers to paraphrased queries [7] and often fail to change their answer under negation [14]. Logical constraints between related questions have been used both to measure and to train for consistency [15], and BECEL collects several consistency types, including negational, symmetric and transitive consistency, into one benchmark [13]. For probabilistic predictions, consistency checks such as negation, monotonicity and Bayes’ rule have been proposed to evaluate models whose correctness cannot be verified directly [8], and extended to forecasting [19]. JevBench applies the same principle to a different object: typed probabilistic decision models behind a fixed interface, whose outputs are full distributions over declared answers, so that additivity, conditioning on a menu and batch independence can be tested directly.

Table 1 | JevBench and the consistency evaluations closest to it. *Gold*: whether the consistency score needs correct answers (a dash: not classified here, since its test cases differ). *Test set*: how the tested inputs are fixed.

Work	What is tested	Laws	Answers compared	Gold	Test set
CheckList [21]	predictions of NLP models on three tasks	capability tests: minimum functionality, invariance, directional expectations	predicted labels	partly	generated per task with a tool
ParaRel [7]	factual knowledge of pretrained LMs	invariance under paraphrase	cloze predictions	no	328 paraphrases of 38 relations
Logic-driven consistency [15]	natural language inference	logic rules over related examples	predicted labels	no	built from NLI data
BECEL [13]	pretrained LMs on downstream tasks	several types of behavioural consistency (19 test cases)	task predictions	–	fixed benchmark
Consistency checks [8]	chess engines, forecasters, legal judges	logical, human-interpretable rules per task	valuations, forecasts, decisions	no	generated per task
Forecaster consistency [19]	LLM forecasters	logical relations between questions, scored by arbitrage	forecast probabilities	no	generated question sets
JevBench	typed decision models behind one interface	50 laws of representation, batch independence, additivity, logical order and choice-set conditioning	full distributions over yes/no, options or levels	no	frozen requests, one plan for every model

Positioning. Table 1 places JevBench among these evaluations. Three choices set it apart from them. First, it compares full distributions over typed answers, including Choices over several options and ordered Scores, rather than labels or the probability of a single proposition. This is what makes laws testable that have no counterpart for labels: additivity under merging, splitting and coarsening of options, the marginal consistency of a Noul with the Choice it summarises, and conditioning on a changed menu, Luce’s choice axiom [17]. Second, it treats the request as part of the input: a decision model answers a batch of questions at once, and batch independence is one of its five dimensions. Third, every test and every request is frozen in advance and judged in one unit, probability, at one tolerance, so that models of any architecture are scored on the same tests and their scores are comparable. Like the consistency checks of [8, 19], JevBench needs no correct answers; [19] found that their consistency metrics correlate with forecasters’ later Brier scores, and we report coherence beside accuracy for the same reason (§5.3).

Sensitivity to surface form and batching. Large language models are sensitive to the order of answer options [20], prefer some option identifiers over others [28], and change their accuracy with semantically irrelevant changes of prompt format [22]. Answering several questions in one prompt reduces cost [4] but lets the questions influence each other. JevBench’s representation and batch dimensions test these effects as laws: option order and identifiers are relabellings with known effect, and a question’s answer must not depend on its companions.

Coherence of probability and choice. A coherent assignment of probabilities is one that admits no sure loss, which forces the additivity laws of a probability measure [5]; human probability judgments violate the conjunction rule [26], whose bounds go back to Fréchet [9]. Luce’s choice axiom [17] requires choice probabilities to be independent of irrelevant alternatives and implies regularity [1]; its well-known failures for similar alternatives [25] and decoys [12, 24] determine which menu changes JevBench tests with conditioning and which with the partition law (Appendix A.3).

Accuracy and calibration. Calibration measures whether predicted probabilities match observed frequencies [11], and proper scoring rules such as the Brier score [2] reward accurate probabilities; holistic evaluations combine many such metrics across scenarios [16]. All of these need correct answers. JevBench needs none, and we use it beside accuracy rather than instead of it (§5.3).

3 Method

JevBench tests whether a typed probabilistic decision model answers related questions coherently. It never asks whether an answer is correct: every test compares the model with itself, under a transformation of the question whose effect on the answer is known in advance. This section defines the setting (§3.1), the relations

(§3.2), how their violations are measured (§3.3), the scores (§3.5), and the oracle families that organise the 50 relations (§3.7–3.8).

3.1 Setting

Let $\mathcal{S}$ denote the state space and $\mathcal{Q}$ the space of typed questions. A question $q \in \mathcal{Q}$ consists of a key, instructions, criteria and an answer type, and is posed together with the other questions of its request; these companions are part of q . Each question has a finite answer space $\mathcal{Y}_q$: $\{0, 1\}$ for a Noul, the set of option identifiers for a Choice, and the ordered levels $\{0, \dots, L-1\}$ for a Score. A Choice forces an answer among its options: its output is the model's distribution over them given that the answer is one of them, and a Score likewise over its levels. For a finite answer space $\mathcal{Y}$,

$$\Delta(\mathcal{Y}) = \left\{ P : \mathcal{Y} \rightarrow [0, 1] \mid \sum_{y \in \mathcal{Y}} P(y) = 1 \right\} \quad (1)$$

is the probability simplex over $\mathcal{Y}$. A Jev model is a typed probabilistic decision function

$$f : \mathcal{S} \times \mathcal{Q} \rightarrow \bigcup_{q \in \mathcal{Q}} \Delta(\mathcal{Y}_q), \quad f(s, q) \in \Delta(\mathcal{Y}_q). \quad (2)$$

We write $P_q = f(s, q)$ for the model's answer to q under a state s that is clear from context, and $p_q = P_q(1)$ for the probability of yes of a Noul.

3.2 Metamorphic relations

Definition 1 (Metamorphic relation). *A JevBench metamorphic relation is a triple*

$$r = (A_r, T_r, \mathcal{R}_r), \quad (3)$$

where

1. $A_r : \mathcal{S} \times \mathcal{Q} \rightarrow \{0, 1\}$ is the applicability indicator: $A_r(s, q) = 1$ when the construction of the relation is valid for the case (s, q) ;
2. T_r is a transformation that maps a source question q to related questions

$$T_r(q) = (q'_1, \dots, q'_{m_r}), \quad (4)$$

all answered under the same state, with the joint output $\mathbf{P} = (P_q, P_{q'_1}, \dots, P_{q'_{m_r}})$;

3. $\mathcal{R}_r$ is the oracle: a single law that the joint output must satisfy, in one of three forms,

$$(E) \text{ distributional equality:} \quad \Phi_r(\mathbf{P}) = \Psi_r(\mathbf{P}), \quad (5)$$

$$(I) \text{ identity:} \quad g_r(\mathbf{P}) = 0, \quad (6)$$

$$(L) \text{ inequality:} \quad g_r(\mathbf{P}) \leq 0. \quad (7)$$

In (E), Φ_r and Ψ_r map the joint output to two distributions over a common answer space, by pushforward along a known map of answers or by renormalisation to a subset of options; in (I) and (L), g_r is a real-valued function of the joint output. These maps are determined by the questions alone.

A model f is consistent with r at a case (s, q) with $A_r(s, q) = 1$ if its joint output satisfies $\mathcal{R}_r$.

One relation, one law. Each relation states exactly one law, so relations and laws correspond one to one. Two laws about the same transformed questions are two relations that share a transformation. A law may be stated for every element i of a finite index fixed by the question, such as every option a , every level k or every batch size. It then remains one law, whose instances all have the same form, and it holds exactly when every instance does.

Most relations need a single related question ($m_r = 1$). Relations over several propositions use more. Four properties hold by construction.

- (P1) *Fixed state.* All questions of a test are answered under the state s of the source case; only the question side is transformed.

- (P2) *No ground truth.* The oracle refers only to the model's own outputs, through maps fixed by the questions. It never refers to the correct answer of the decision problem, and neither does the applicability indicator.
- (P3) *Model agnosticism.* The definition uses the model only through f , so it applies unchanged to any architecture that implements the interface.
- (P4) *Fixed test set.* The transformation and the applicability indicator depend on the case alone, never on the model's outputs, so every model answers exactly the same questions and all relation scores are computed on the same tests.

What coherence does not measure. By (P2), a relation holds or fails whatever the correct answer is: JevBench measures whether a model's answers are consistent with one another, not whether they are right. Coherence is therefore necessary for a trustworthy decision model but not sufficient. A model that ignores its input and returns a uniform distribution over each question's declared answers satisfies every invariance and many additive laws by construction and scores high, while deciding nothing. Coherence scores should be read beside an accuracy or calibration evaluation of the same model, never instead of one.

Examples. The following examples, all on a customer-support case, show the three forms.

Example 1 (Form (E): option order). Let q be the Choice "Which team should own this request?" with options billing, engineering, retention, and let $q' = T_r(q)$ list the same options in reverse order under the neutral identifiers x, y, z . With $\sigma : \text{billing} \mapsto z, \text{engineering} \mapsto y, \text{retention} \mapsto x$, the oracle is

$$P_{q'} = \sigma_{\#} P_q, \quad \text{that is} \quad P_{q'}(\sigma(o)) = P_q(o) \text{ for every option } o, \quad (8)$$

with $\Phi_r(\mathbf{P}) = P_{q'}$ and $\Psi_r(\mathbf{P}) = \sigma_{\#} P_q$.

Example 2 (Form (I): complement). Let q be the Noul "Does the customer ask for a refund?" and q' its negation "Is it the case that the customer does not ask for a refund?". The oracle is the complement law

$$g_r(\mathbf{P}) = p_q + p_{q'} - 1 = 0. \quad (9)$$

The cumulative relation of a Score is an identity stated for every level: with q'_k the Noul "Is the level k or higher?", the law is $g_{r,k}(\mathbf{P}) = p_{q'_k} - \sum_{j \geq k} P_q(j) = 0$ for every $k \geq 1$.

Example 3 (Form (L): conjunction bound). Let q and q_B be the Nouns "Is a feature failing?" (A) and "Did the customer change their configuration?" (B), and let $q_{A \wedge B}$ ask whether both hold, so that $T_r(q) = (q_B, q_{A \wedge B})$. A conjunction is never more probable than either conjunct [9]:

$$g_r(\mathbf{P}) = p_{A \wedge B} - \min\{p_A, p_B\} \leq 0. \quad (10)$$

One transformation may also carry two laws, and then gives two relations. Duplicating an option o as o' requires both that the two copies together keep the mass of o , $\phi_{\#} P_{q'} = P_q$ with $\phi(o') = o$, and that they share it equally, $P_{q'}(o) = P_{q'}(o')$.

3.3 Measuring violations

An oracle is an exact statement, whereas a model returns real-valued probabilities: its outputs rarely satisfy an equality exactly, and a law missed by 10^{-6} is not broken in the way one missed by 0.5 is. To measure how far the outputs under a transformation depart from the law, JevBench assigns each relation a violation measure determined by the form of its oracle.

Definition 2 (Violation measure). *The violation measure of relation r is*

$$\delta_r(\mathbf{P}) = \begin{cases} \text{TV}(\Phi_r(\mathbf{P}), \Psi_r(\mathbf{P})) & \text{for form (E),} \\ |g_r(\mathbf{P})| & \text{for form (I),} \\ \max\{0, g_r(\mathbf{P})\} & \text{for form (L),} \end{cases} \quad (11)$$

where $\text{TV}(P, Q) = \frac{1}{2} \sum_y |P(y) - Q(y)|$ is the total variation distance.

For a law stated over an index, the measure is the largest violation among its instances,

$$\delta_r(\mathbf{P}) = \max_i \delta_{r,i}(\mathbf{P}), \quad (12)$$

with each $\delta_{r,i}$ given by (11); for the cumulative law of Example 2, $\delta_r = \max_k |g_{r,k}(\mathbf{P})|$.

Proposition 1. For every relation, $\delta_r(\mathbf{P}) \geq 0$, and

$$\delta_r(\mathbf{P}) = 0 \iff \mathbf{P} \text{ satisfies } \mathcal{R}_r. \quad (13)$$

Proof. Each case of (11) is nonnegative. $\text{TV}(P, Q) = 0$ if and only if $P = Q$; $|g| = 0$ if and only if $g = 0$; and $\max\{0, g\} = 0$ if and only if $g \leq 0$. For a law over an index, the maximum (12) of nonnegative terms is zero exactly when every term is, that is, when every instance holds. $\square$

The three measures share one reading: each is the largest amount by which the law fails, in probability units. For (I) and (L) this is direct. For (E) it follows from $\text{TV}(P, Q) = \max_{E \subseteq \mathcal{Y}} |P(E) - Q(E)|$: the total variation is the largest disagreement between the two distributions on any event. One tolerance is therefore meaningful across all relations.

Example 4 (Measuring the examples). Suppose the model answers the three examples as follows.

- Example 1: $P_q = (0.70, 0.20, 0.10)$ for billing, engineering, retention, and $0.62, 0.24, 0.14$ for their images z, y, x . Then $\delta_r = \frac{1}{2}(0.08 + 0.04 + 0.04) = 0.08$.
- Example 2: $p_q = 0.80$ and $p_{q'} = 0.25$. Then $\delta_r = |0.80 + 0.25 - 1| = 0.05$.
- Example 3: $p_A = 0.60, p_B = 0.70, p_{A \wedge B} = 0.66$. Then $\delta_r = \max\{0, 0.66 - 0.60\} = 0.06$.

Tolerance. A test (s, q) with $A_r(s, q) = 1$ passes relation r at tolerance $\varepsilon \geq 0$ if

$$\delta_r(\mathbf{P}) \leq \varepsilon, \quad (14)$$

and its violation indicator is $V_r(s, q; \varepsilon) = \mathbb{I}[\delta_r(\mathbf{P}) > \varepsilon]$. The current implementation uses $\varepsilon = 0.05$ for all relations and all models. In Example 4, the permuted Choice (0.08) and the conjunction bound (0.06) fail, while the complement (0.05) passes.

Proposition 2. For every relation and test,

1. the test passes at $\varepsilon = 0$ if and only if the model is consistent with r ;
2. if the test passes at ε , it passes at every $\varepsilon' \geq \varepsilon$;
3. the test passes at every $\varepsilon > 0$ if and only if the model is consistent with r .

Proof. (1) follows from Proposition 1, since $\delta_r \geq 0$, and (2) is immediate. For (3), $\delta_r(\mathbf{P}) \leq \varepsilon$ for all $\varepsilon > 0$ exactly when $\delta_r(\mathbf{P}) = 0$. $\square$

The tolerance thus relaxes each law monotonically and recovers it exactly in the limit.

Relation score. Let $(s_1, q_1), \dots, (s_N, q_N)$ be the cases of the benchmark, and let

$$N_r = \sum_{i=1}^N A_r(s_i, q_i) > 0 \quad (15)$$

be the number of tests of relation r . The relation score is the pass rate over these tests,

$$S_r(\varepsilon) = 1 - \frac{1}{N_r} \sum_{i=1}^N A_r(s_i, q_i) V_r(s_i, q_i; \varepsilon) \in [0, 1]. \quad (16)$$

By Proposition 2, $S_r(\varepsilon)$ is nondecreasing in ε , and $S_r(0)$ is the fraction of tests on which the model is exactly consistent with r .

The frozen benchmark. A benchmark fixes, once and for all, one test of every relation on every case, so that a benchmark of N cases has exactly N tests of each relation, each one request. When the instances of a law are all answered in one request (the options of a Choice, the levels or tails of a Score, the thresholds of a count, the questions of a reordered batch), the test measures the maximum (12) over them. When each instance needs a request of its own (a template, a removed option, an option pair, a batch size), the test takes one instance i , and measures $\delta_{r,i}$. That instance is not drawn at random: within each domain, the tests of a relation take its instances in turn, so that every instance is used as evenly as possible. Since $\delta_{r,i} \leq \delta_r$, a violated test is still a

violated law; for such relations S_r estimates the fraction of (case, instance) pairs at which the law holds, which is never lower, and may be higher, than the fraction of cases at which every instance holds. The tests, their requests and this choice are frozen with the cases and are the same for every model.

The reported benchmark, JevBench-mini, is a fixed subset of these tests with the same number of tests for every relation (§4.3).

Summary. Definition and measurement combine into the basic JevBench testing unit,

$$q \xrightarrow{T_r} (q'_1, \dots, q'_{m_r}) \xrightarrow{f(s, \cdot)} \mathbf{P} \xrightarrow{\delta_r} \delta_r(\mathbf{P}) \xrightarrow{\leq \varepsilon?} V_r \quad (17)$$

The relation $(A_r, T_r, \mathcal{R}_r)$ states one law of coherence; the form of its oracle fixes the measure δ_r ; and the tolerance ε decides how far from the law a test may be and still pass.

3.4 Estimating the source output

The source output P_q is the model's answer to one request of the case's questions, and each follow-up output $P_{q'}$ is likewise the answer to one request, so a test compares one answer with one answer. No model we served ourselves varied: three identical requests returned identical answers for all 22 self-hosted models under their tested serving configurations (repeat noise 0). For a model that samples, a single request is one draw of P_q , and its sampling noise enters δ_r like any other deviation; the same fixed tolerance is used for every model, so a noisy model gains no slack. As a diagnostic outside the benchmark, JevBench can request a case R times and report the repeat noise

$$v_q = \max_{t \leq R} \text{TV}(P_q^{[t]}, \bar{P}_q), \quad \bar{P}_q = \frac{1}{R} \sum_{t=1}^R P_q^{[t]}. \quad (18)$$

Of the five hosted models we tested this way (three requests of each of 28 questions), three returned identical answers and two showed mean repeat noise 0.002 and 0.010, well under the tolerance.

3.5 Scores

Which relations are scored. Let $\mathfrak{R}$ be the set of 50 relations (§3.8). For a given model, a relation enters the scores only if it has at least $n_{\min} = 5$ applicable tests. We write $\mathfrak{R}^* \subseteq \mathfrak{R}$ for the scored relations and $\mathfrak{R}_d^*$ for those in dimension d .

Dimension and overall score. A dimension score is the mean of its relation scores,

$$S_d = \frac{1}{|\mathfrak{R}_d^*|} \sum_{r \in \mathfrak{R}_d^*} S_r \quad (19)$$

and the overall score weights the five dimensions $\mathcal{D} = \{\text{REP}, \text{BAT}, \text{MEA}, \text{LOG}, \text{CHO}\}$ equally,

$$S_{\text{JevBench}} = \frac{1}{|\mathcal{D}|} \sum_{d \in \mathcal{D}} S_d. \quad (20)$$

A dimension with no scored relation is reported as missing, and the overall score of that model is not reported.

Coverage. A model may refuse a request or answer it invalidly, for example when a server limits the number of questions per request. Such tests are not judged, and a model that skipped the tests it fails would score higher on the rest. A relation is therefore scored only when at least 95% of its planned tests were judged; otherwise the relation, its dimension and the overall score are reported as missing, so that every model is scored on the same tests or not at all.

Remark 1. Groups (§3.8) serve interpretation and reporting, and group scores are the mean of their relation scores. They add no weighting level to (19): regrouping the relations inside a dimension changes neither S_d nor S_{JevBench} . Weighting the dimensions equally, rather than the relations, keeps a dimension's weight independent of how many relations it happens to contain.

Uncertainty. Instances from one case share a state and are not independent. We therefore resample cases with replacement (1,000 draws, fixed seed), recompute every score on each draw, and report the 2.5th and 97.5th percentiles as a 95% interval.

Scoring at a glance

1. *Test.* One relation on one case: the source answer P_q comes from the case's base request, each follow-up from one request; δ_r is the total variation (form E), $|g_r|$ (I) or $\max\{0, g_r\}$ (L), the largest over the instances answered in one request.
2. *Pass.* $\delta_r \leq 0.05$, the same tolerance for every relation and model.
3. *Relation score S_r :* the share of its tests that pass (24 per relation on JevBench-mini, 240 on JevBench-240).
4. *Dimension score S_d :* the mean of its relations' scores (REP 15, BAT 5, MEA 17, LOG 10, CHO 3 relations); groups only report.
5. *Overall score:* the mean of the five dimension scores.
6. *Not scored:* a relation with fewer than five tests or under 95% coverage; a dimension or overall score that would need it.
7. *Interval:* 1,000 bootstrap draws of the cases.

Graded score. The binary score counts violations but not their size. As a secondary measure we also report, for each relation, one minus the mean excess $\max\{0, \delta_r - \varepsilon\} / (1 - \varepsilon)$, aggregated in the same way.

3.6 Semantic mappings and pushforward distributions

Several relations compare distributions over different answer spaces whose outcomes are related by a known map. For $\phi : \mathcal{Y} \rightarrow \mathcal{Y}'$ and $P \in \Delta(\mathcal{Y})$, the pushforward is

$$(\phi_{\#}P)(y') = \sum_{y \in \mathcal{Y}: \phi(y)=y'} P(y). \quad (21)$$

A bijective ϕ relabels the probability vector; a many-to-one ϕ sums the masses of the outcomes it merges. Permutation, renaming, answer-type encoding, merging, coarsening and marginalisation are all instances of (21).

3.7 Oracle families

JevBench classifies relations by the operation that links the two distributions its oracle compares, not by the part of the request the transformation edits. Five operations give five dimensions (Table 2). Transformations that edit different parts of a question share a dimension when their oracles use the same operation, and transformations that edit the same part may fall in different dimensions. We write P_q for a source output and $P_{q'}$ for the corresponding follow-up output; for each family we give the oracle and its violation measure (§3.3).

Table 2 | The five dimensions and the operations that define them.

Dimension	Operation linking the two distributions	Form	What changes
REP	bijection σ of answers	E	the description of the same events
BAT	identity	E	the other questions of the request
MEA	many-to-one map ϕ of outcomes, additive identity	E, I	the events of one measure
LOG	order	L	the propositions, ordered by entailment
CHO	restriction to a sub-menu	E	the menu a Choice is conditioned on

REP: representation consistency. Two questions describe the same events, and their outputs must correspond through the bijection σ between their answer spaces:

$$P_{q'} = \sigma_{\#}P_q, \quad \delta = \text{TV}(P_{q'}, \sigma_{\#}P_q). \quad (22)$$

The four groups differ only in σ . *Invariance*: no answer changes its meaning, so $\sigma = \text{id}$: a paraphrase, a translation, typographical noise, an added clarification, a neutral key, a paraphrased option description, and logically equivalent propositions such as $\neg(A \wedge B)$ and $\neg A \vee \neg B$. Paraphrases, translations and natural negations are generated once, judged, reviewed and frozen with the cases; each test uses exactly one of them, the first usable rewrite of its source in the frozen bank, so every test of a relation has the same form. *Equivariance*: σ is a known relabelling of the answers (options reordered, Score levels reversed with $\sigma(k) = L - 1 - k$, option identifiers renamed); a question with two identical options is mapped to itself by the relabelling τ that exchanges them, so $P_{q'} = \tau_{\#}P_q$. *Binding*: a surface cue (a question key or an option identifier) suggests a correspondence that differs from the semantic one, and the output must follow σ : a negated or swapped key leaves the meaning intact, so $\sigma = \text{id}$, and exchanging two options' identifiers or descriptions makes σ the transposition that follows the descriptions. *Type*: two answer types encode the same random variable, and σ is the encoding, for example $\{0, 1\} \rightarrow \{\text{no}, \text{yes}\}$ between a Noul and a two-option Choice; for binary answers the total variation is the gap in the probability of yes.

BAT: batch independence. A question is posed together with the other questions of its case. The follow-up q' is the same question posed with different companions (alone, in reversed order, with replaced companions, in batches of other sizes, or twice), and its output must not change:

$$P_{q'} = P_q, \quad \delta = \text{TV}(P_{q'}, P_q). \quad (23)$$

MEA: probability measure coherence. The questions ask about different events of one probability space, and the outputs must be compatible with a finitely additive probability measure μ , with $\mu(\Omega) = 1$ and $\mu(\biguplus_i A_i) = \sum_i \mu(A_i)$. The operation is a many-to-one map of outcomes, or the additive identity it induces. *Complement*: for an event A asked as two separate questions, $\delta = |p_A + p_{A^c} - 1|$, where p is the probability of yes. *Partition*: within one question, the follow-up options coarsen or refine the source options through a map ϕ that sends each fine outcome to the coarse outcome containing it. For a coarsening (merge, hierarchy, adjacent Score levels merged) $\delta = \text{TV}(P_{q'}, \phi_{\#}P_q)$; for a refinement $\delta = \text{TV}(\phi_{\#}P_{q'}, P_q)$. Refinements split an option by a condition, duplicate it exactly or in other words, or add a level beyond the top of a Score, which refines the open-ended top level, so that ϕ maps the new level to the old top. Replacing an option o by a catch-all c , “none of the other options”, relabels the event o , so $\phi(o) = c$. *Marginalisation*: a question whose answer is a deterministic function π of other questions' outcomes must receive the pushforward $\pi_{\#}P$. For the cumulative Noul “level k or higher” of a Score, $\pi_k(y) = \mathbb{I}[y \geq k]$ and

$$p_{\geq k} = (\pi_{k\#}P_q)(1) = \sum_{j \geq k} P_q(j), \quad \delta = \max_k |p_{\geq k} - \sum_{j \geq k} P_q(j)|. \quad (24)$$

For the Noul “is it o ?” of a Choice, $\pi_o(y) = \mathbb{I}[y = o]$, so its answer must equal $P_q(o)$ for every option. For a four-way Choice over the pairs of answers to A and B , the projections onto each coordinate must equal the separate answers. And the answers to A , B , $A \wedge B$ and $A \vee B$ must be projections of one joint distribution of (A, B) , which requires inclusion–exclusion, $p_{A \wedge B} + p_{A \vee B} = p_A + p_B$.

LOG: logical coherence. The questions are propositions ordered by entailment, and their probabilities must respect the order; every oracle is an inequality of form (L), measured by its excess. *Bounds*: a conjunction is never more probable than either conjunct [9], $p_{A \wedge B} \leq \min\{p_A, p_B\}$; with inclusion–exclusion this gives every Fréchet bound of two propositions. For $n \geq 3$ propositions no identity links the conjunction and the disjunction, and the four bounds

$$\max\left\{0, \sum_i p_{A_i} - (n-1)\right\} \leq p_{\bigwedge_i A_i} \leq \min_i p_{A_i}, \quad (25)$$

$$\max_i p_{A_i} \leq p_{\bigvee_i A_i} \leq \min\left\{1, \sum_i p_{A_i}\right\} \quad (26)$$

are four laws. For “at least two of $A_1, \dots, A_n$ ” with $n \geq 3$, the counting bounds $\frac{1}{n-1}(\sum_i p_{A_i} - 1) \leq p_{\geq 2} \leq \frac{1}{2} \sum_i p_{A_i}$ follow from $\mathbb{E}[N] = \sum_i p_{A_i}$ for the number N of true propositions. *Monotonicity*: for a chain $A_1 \models A_2 \models \dots \models A_n$ (a stricter threshold, a stronger claim, a stronger quantifier),

$$p_{A_1} \leq p_{A_2} \leq \dots \leq p_{A_n}, \quad \delta = \max_{i < j} \max\{0, p_{A_i} - p_{A_j}\}. \quad (27)$$

CHO: choice-set coherence. A Choice is the model’s distribution over its options given that the answer is one of them (§3.1). Changing the menu from C to C' therefore changes the event conditioned on. With $K = C \cap C'$ the options the two menus share and $P|_K$ a distribution renormalised to K , conditioning on K within either menu must give the same result:

$$P_{q'}|_K = P_q|_K, \quad \delta = \text{TV}(P_{q'}|_K, P_q|_K). \quad (28)$$

This is Luce’s independence of irrelevant alternatives [17], which implies regularity [1]; comparing renormalised distributions keeps δ bounded. It is tested when an off-topic option is added, when each option in turn is removed, and when each pair of options is asked alone, the binary form of the choice axiom. The law needs the added or removed options to be events disjoint from the others. An option that overlaps an existing one, such as a near-duplicate or a level beyond the top of a Score, refines that option instead, and its law is the partition law of MEA.

Boundaries. REP compares two descriptions of the same events; MEA compares different events of one measure under the same menu; CHO compares the same events under different menus; LOG orders probabilities rather than equating them; and BAT keeps the question fixed and changes only its companions.

3.8 Taxonomy of the relations

Table 3 lists the 50 relations by dimension and group, each with its transformation, its law and the form of the law, which fixes its violation measure (Definition 2). Each relation is placed by the operation of its oracle, so its position follows from the definition of its dimension. The taxonomy is fixed in advance and does not depend on how any tested model behaves. The relations are irredundant: no relation’s law follows from the laws of the other relations on the same outputs. Appendix A lists the further laws we considered and the reason each is not a relation.

Table 3 | The 50 relations. P_q : source output; $P_{q'}$: follow-up output; p : probability of yes of a Noul; C : the original options; $P|_K$: a distribution renormalised to K ; σ , ϕ , π : known maps of answers. Form: (E) distributional equality, (I) identity, (L) inequality; a law with “for every” is stated over an index (§3.2).

Group	Relation	Form	Transformation	Law
REP Representation consistency (15)				
invariance	instruction_paraphrase	E	judged paraphrase of the instructions	$P_{q'} = P_q$
	cross_lingual	E	question translated	$P_{q'} = P_q$
	typo_noise	E	typos, casing, punctuation	$P_{q'} = P_q$
	verbosity	E	redundant clarification added	$P_{q'} = P_q$
	key_rename	E	neutral question key	$P_{q'} = P_q$
	description_paraphrase	E	one option description paraphrased	$P_{q'} = P_q$
	de_morgan	E	Nouls q'_1 : $\neg(A \wedge B)$ and q'_2 : $\neg A \vee \neg B$	$P_{q'_1} = P_{q'_2}$
equivariance	option_permutation	E	options reordered; Score levels reversed	$P_{q'} = \sigma_{\#} P_q$
	option_rename	E	neutral option identifiers	$P_{q'} = \sigma_{\#} P_q$
	clone_symmetry	E	option o duplicated as o'	$P_{q'} = \tau_{\#} P_{q'}, \tau$ swaps o, o'
binding	key_instruction_conflict	E	key negated, or swapped with another question’s	$P_{q'} = P_q$
	name_description_conflict	E	two options’ identifiers exchanged; descriptions keep their positions	$P_{q'} = \sigma_{\#} P_q, \sigma$ follows the descriptions
	description_swap	E	two options’ descriptions exchanged; identifiers keep their positions	$P_{q'} = \sigma_{\#} P_q, \sigma$ follows the descriptions
type	noul_as_choice	E	Noul asked as a yes/no Choice	$P_{q'} = \sigma_{\#} P_q$
	noul_as_score	E	Noul asked as a two-level Score	$P_{q'} = \sigma_{\#} P_q$
BAT Batch independence (5)				
isolation	batch_solo	E	q asked alone	$P_{q'} = P_q$
	batch_order	E	batch order reversed	$P_{q'} = P_q$

Group	Relation	Form	Transformation	Law
	duplicate_question	E	q asked twice in one batch, as q'_1, q'_2	$P_{q'_1} = P_{q'_2}$
	batch_content	E	companions replaced, batch size kept	$P_{q'} = P_q$
	batch_size_sweep	E	batches q'_n of the first 2, 5 and 10 questions	$P_{q'_n} = P_q$ for every n
MEA Probability measure coherence (17)				
complement	negation_wrapper	I	templated negation q' of q	$p_q + p_{q'} - 1 = 0$
	natural_negation	I	natural opposite q' of q	$p_q + p_{q'} - 1 = 0$
	choice_complement	I	Noul q' : "not o "	$p_{q'} + P_q(o) - 1 = 0$
	score_complement	I	Noul q' : "not level k "	$p_{q'} + P_q(k) - 1 = 0$
partition	compound_negation	I	Nouls $A \wedge B$ and $\neg(A \wedge B)$	$p_{\neg(A \wedge B)} + p_{A \wedge B} - 1 = 0$
	option_merge	E	options a, b merged	$P_{q'} = \phi_{\#} P_q$
	option_split	E	option o split by a condition X	$\phi_{\#} P_{q'} = P_q$
	option_clone	E	option o duplicated	$\phi_{\#} P_{q'} = P_q$
	similarity_effect	E	near-duplicate b' of b added, in other words	$\phi_{\#} P_{q'} = P_q, \phi(b') = b$
	partition_hierarchy	E	options grouped into coarse blocks	$P_{q'} = \phi_{\#} P_q$
	score_granularity	E	adjacent Score levels merged	$P_{q'} = \phi_{\#} P_q$
	extreme_level	E	level beyond the top of a Score added	$\phi_{\#} P_{q'} = P_q, \phi$ maps the new level to the top
marginalisation	option_closure	E	each option o in turn replaced by a catch-all c , "none of the other options"	$P_{q'_o} = \phi_{o\#} P_q, \phi_o(o) = c$, for every o
	score_cumulative	I	Nouls q'_k : "level k or higher"	$p_{q'_k} - \sum_{j \geq k} P_q(j) = 0$ for every k
	choice_indicator	I	one Noul q'_o : "is it o ?" per option	$p_{q'_o} - P_q(o) = 0$ for every o
	pair_choice_marginals	I	four-way Choice q' over the answers to A, B	$\pi_{i\#} P_{q'}(1) - p_i = 0$ for every $i \in \{A, B\}$
	inclusion_exclusion	I	Nouls $A \wedge B$ and $A \vee B$	$p_{A \wedge B} + p_{A \vee B} - p_A - p_B = 0$
LOG Logical coherence (10)				
bounds	frechet_and_upper	L	Noul $A \wedge B$	$p_{A \wedge B} - \min\{p_A, p_B\} \leq 0$
	nary_and_upper	L	Noul $\bigwedge_i A_i, n \geq 3$	$p_{\bigwedge} - \min_i p_{A_i} \leq 0$
	nary_and_lower	L	Noul $\bigwedge_i A_i, n \geq 3$	$\sum_i p_{A_i} - (n-1) - p_{\bigwedge} \leq 0$
	nary_or_upper	L	Noul $\bigvee_i A_i, n \geq 3$	$p_{\bigvee} - \sum_i p_{A_i} \leq 0$
	nary_or_lower	L	Noul $\bigvee_i A_i, n \geq 3$	$\max_i p_{A_i} - p_{\bigvee} \leq 0$
	count_upper	L	Noul "at least two of $A_1, \dots, A_n$ ", $n \geq 3$	$p_{\geq 2} - \frac{1}{2} \sum_i p_{A_i} \leq 0$
	count_lower	L	Noul "at least two of $A_1, \dots, A_n$ ", $n \geq 3$	$\frac{\sum_i p_{A_i} - 1}{n-1} - p_{\geq 2} \leq 0$
monotonicity	threshold_sweep	L	Nouls "more than k " for a grid of k	$p_{>k'} - p_{>k} \leq 0$ for every $k < k'$
	entailment_strength	L	a claim with a stronger and a weaker version	$p_A - p_B \leq 0$ for every $A \models B$
	quantifier_monotonicity	L	"all", "at least two", "at least one"	$p_A - p_B \leq 0$ for every $A \models B$
CHO Choice-set coherence (3)				
conditioning	irrelevant_option	E	off-topic option z added	$P_{q'} _C = P_q$
	remove_option_ii	E	each option o in turn removed	$P_{q'_o} = P_q _{C \setminus \{o\}}$ for every o
	pairwise_ii	E	each pair $\{a, b\}$ of options asked alone, $k \geq 4$ options	$P_{q'_{ab}} = P_q _{\{a, b\}}$ for every pair

Dependence on answer-type equivalence. Four MEA relations compare a Noul with a Choice or Score answer: choice_complement, score_complement, score_cumulative and choice_indicator. They assume, beyond additivity, that the two answer types encode the same event, the property REP.type tests. A model that fails REP.type will tend to fail these relations as well; we report them in MEA because the constraint they check is additive.

Table 4 | The twelve domains of JevBench - 240: distinct question keys, questions by type over the domain’s 20 cases, and the mean length of a state.

Domain	Question keys	Nouls	Choices	Scores	Words per state
Customer support	14	160	60	20	154
Content moderation	15	160	40	40	198
Finance operations	19	110	46	34	200
Security operations	17	115	60	30	195
IT incidents	16	98	54	20	190
E-commerce returns	13	114	60	20	156
HR recruiting	16	105	40	20	181
Healthcare administration	21	95	45	20	205
Legal contracts	19	105	50	20	174
Logistics	14	90	60	20	191
Education grading	15	80	60	20	176
AI agent logs	14	109	40	20	171
All	193	1341	615	284	183

4 The benchmark

A relation needs cases to act on: a state, and a batch of typed questions about it. This section describes how the cases were made (§4.1), the frozen bank of natural rewrites that four relations use (§4.2), the frozen test plans (§4.3) and the protocol of a run (§4.4).

4.1 Cases

JevBench-240 contains 240 synthetic cases, 20 in each of 12 decision domains (Table 4): customer support, content moderation, finance, security and IT operations, e-commerce returns, recruiting, healthcare administration, legal contracts, logistics, grading and the logs of AI agents. A case has a state of 66–382 words (183 on average) and 8–12 questions about it, 2,240 in all: 1,341 Nouls, 615 Choices (four options each) and 284 Scores (four or five levels).

The cases are generated *fact-first*, so that every question receives a rule-based gold annotation, including an explicit unknown label when the facts it needs are absent, and the data are balanced by construction. A domain is specified by hand as a set of fact slots, four scenario archetypes, a bank of questions, each with a rule that computes its answer from the facts, and a set of text styles (for example a customer email, a live chat transcript or a moderation queue entry). Generation then proceeds in four steps.

1. *Plan* (no model). Facts are sampled for an archetype, together with how each is mentioned: stated explicitly, implied by context, or absent from the text. Every gold answer is computed from the facts. Archetypes, gold answers, the position of the correct option of a Choice, styles and three length bands are balanced greedily over a large pool of plans.
2. *Write*. A writer model (gpt-oss-20b [18]) turns the facts into a state in the planned style and length. It never sees the questions.
3. *Verify*. A verifier from another model family (Llama-3.1-8B-Instruct [10]) reads only the state and reports each planned fact, or that it is not stated. Any mismatch with the plan rejects the draft.
4. *Freeze*. Length and near-duplicate checks; 20 cases are kept per domain.

Questions of the form “does the text say *X*” are false when *X* is absent, while questions of the form “is *X* true” have no answer when *X* is absent; their gold is recorded as unknown. Of the 2,240 gold answers, 302 (13.5%) are unknown and 1,938 are known. Neither the writer nor the verifier belongs to a model family under test, and the gold answers and facts are stored apart from the cases: a run reads only the states and the questions.

The cases are synthetic and in English, and their gold answers are only as good as the plans and the verifier. Coherence does not depend on them, since no relation refers to a correct answer; the gold answers are used only for the accuracy we report beside coherence (§5.3).

Spot check. An independent model, GPT-5.6 Sol, answered 48 questions of 24 cases drawn from JevBench-mini with a fixed seed (two cases per domain, a yes/no question and a Choice or Score from each) without

seeing the gold answers, and judged 40 of the natural rewrites that JevBench-mini uses (ten of each of the four kinds). It agreed with the gold answer on 85.4% of the questions (95% interval 72.8–92.8%): 23/24 yes/no questions, 12/12 Choices and 6/12 Scores. We reviewed all 8 disagreements by hand and judged each to stem from natural semantic vagueness rather than a definite error of either side. Most concern Scores, whose gold level follows from a rule over the planned facts: when an input of the rule is absent from the text the gold is unknown, although a reader may still infer a level, and a reader may weigh cues the rule does not use (an expense claim’s amount, say, where the rule looks only at the receipt). Such vagueness bears on the accuracy reported beside coherence and on nothing else: no relation refers to a correct answer. The rewrites, which four relations do use, were judged valid in 39 of 40 cases; the exception renders an option that names an action as a noun phrase in the Chinese translation.

4.2 The rewrite bank

Four relations need natural rather than templated rewrites: a paraphrase of the instructions (`instruction_paraphrase`), a translation (`cross_lingual`), a paraphrase of one option description (`description_paraphrase`) and a natural opposite of a Noul (`natural_negation`). They are generated once and frozen with the cases. A generator (gpt-oss-20b) proposed 1,697 rewrites; a judge (the same model, two votes) rejected 529 of them as not meaning-preserving (or, for negations, not the exact opposite); and a second review by another model (Claude) rejected 345 more. The 823 remaining rewrites (294 paraphrases, 181 option descriptions, 180 Chinese translations and 168 negations) cover 441 distinct source texts. Beyond the spot check (§4.1), the bank has not been audited by humans. A test uses exactly one rewrite: the first usable one for its source in the bank’s fixed order, so every test of a relation has the same form.

4.3 Frozen test plans

For every case and every applicable relation, the engine builds candidate trials: the transformed questions, grouped into the requests that will carry them. Building is deterministic, and it is done in a *sealed* context: a relation that tried to read a model output while building a trial would raise an error, so no test can depend on the model (property P4). Each trial is validated (well-formed questions, unique keys, unchanged state) before it is stored.

From 83,964 candidate trials, the plan of JevBench-240 keeps exactly one test per relation and case, 12,000 tests in all. When a relation has several variants for a case (several templates, several options to remove, several batch sizes), the variant is not drawn at random: within each domain the tests of a relation take the variants in turn, so that each is used as evenly as possible (§3.3). The plan stores every request of every trial, 10,320 distinct requests including one base request per case, so that a run only sends them and judges the answers. It draws no random numbers, and settings that would change the tests (another seed, rewrite bank or number of trials) are refused for a bundled suite.

JevBench-mini, the suite we report, keeps the cases and takes a fixed subset of these tests: each relation on two cases of every domain, 24 tests per relation and 1,200 in all, chosen once and frozen. A run sends 1,248 requests, about a ninth of JevBench-240. Every JevBench-mini test is a JevBench-240 test. Section 5.6 measures how closely the two rank models. Each suite records the SHA-256 of its files, and a modified suite is refused.

4.4 Protocol and package

A run sends each case’s questions once, unchanged (the base request), and then every request of the plan. The answers of the base request are the source outputs P_q of every test of that case; each follow-up is the answer to one request. All tests are judged at the fixed tolerance $\varepsilon = 0.05$, a relation is scored from at least five tests and at least 95% coverage (§3.5), and intervals resample cases with replacement (1,000 draws) [6].

The benchmark is distributed as a Python package (Apache-2.0) with no dependency beyond an HTTP client. A model is any server implementing the interface, or any Python function that maps a state and a batch of questions to answers:

```
import jevbench as jb
model = jb.systemone("http://localhost:8000/v1/systemone", model="my-model")
jb.check(model)           # one request: are the answers well formed?
report = jb.evaluate(model) # JevBench-mini
print(report.table("dimension"))
```

`jb.check` sends one request and stops with a message that names the question and the expected format if an answer deviates from the standard one, before a run starts. Requests are cached by model name, so a rerun or a second suite sends only the requests not yet answered. Most servers answer one request at a time, so a run is sped up by replicas of the model, one per GPU; the package keeps exactly one request in flight per replica, which reproduces the sequential answers.

5 Experiments

5.1 Models and setup

We evaluate 21 openly released Jev-compatible models (Table 8 in Appendix B). They span seven architecture families, named by how a model reads its answer: encoders that score one marker per option (Laya, Laya typed-decisions, Von, Verdict); a bi-encoder and a cross-encoder distilled from a decoder teacher (jevhome-E, jevhome-ettin-1b); decoders that read the logits of answer letters (decider-4b and -2b, JevK5, OneJev-4B, Jebadiah 9B, Bespoke-Nimble-9B, JevAny-Qwen direct-token); decoders with a pointer head over the options (Kev-4B, TinyJev-4B, JevAny-Qwen and JevAny-Gemma pointer); decoders with a trained scorer head (Open-Jev-9B, RSI-Jev 2B, OpenThai-SystemOne); and projection heads over frozen contrastive embeddings (CLM). Sizes range from 48M to 9.7B parameters.

Every model was installed from its developers’ release, at a pinned revision, and served with its developers’ own server on one NVIDIA A100 40GB (the two jevhome models run on CPU). Two exceptions are noted in the tables: Bespoke-Nimble-9B ships no locally runnable server, and we serve it with a thin adapter that builds the same request as the developers’ own request compiler; Verdict ships no server, and we use a third-party server that reimplements its inference. A further model, NeoHorse-Jev-4B, was run but is not scored: its server refuses requests with more questions than its own limit, so one relation (`threshold_sweep`) has no answers, and by the coverage rule (§3.5) it has no overall score.

No self-hosted model varied: three identical requests returned identical answers for all 22 models under the tested serving configurations (§3.4). All results are on JevBench-mini; 17 models were also run on JevBench-240 (§5.6).

Reference models. Four models that need no GPU anchor the scale. *Uniform* gives every answer the same probability and ignores its input. *Random* draws a fresh distribution (Dirichlet with unit parameters, a uniform probability for a Noul) for every distinct state and question, seeded by their hash, so it is deterministic but its answers to any two different questions are unrelated. The *Luce toy scorer* scores each option independently by word overlap with the state, and the *biased* variant adds a first-option bias, a dependence on the question key and leakage between the questions of a batch.

Accuracy. Beside coherence, we score the base answers of every model, that is, each case’s questions asked once and unchanged, against the gold answers fixed by the data generation (§4.1), over the 1938 questions whose gold is known. A question counts as correct when its most probable answer is the gold one; a tie of k answers earns $1/k$. This needs no further request.

5.2 Main results

Reading the scores

- *Coherence is not correctness.* A model that ignores its input and answers uniformly scores 89.4, above every model here, at chance accuracy (39.8%); read every score beside the accuracy in the last column (§5.3).
- *Overlapping ranges are ties.* Models whose rank ranges overlap are not separated by the benchmark; on JevBench-mini a difference of a few points is within its intervals (§5.6).
- *Scores are per law.* A relation score is the share of tests whose law holds within 0.05 in probability; the ranking is stable for tolerances from 0.03 to 0.1 (§5.7).

Table 5 lists the results. Overall coherence ranges from 56.5 to 83.9, with a median of 72.2. Open-Jev-9B is first, and its interval separates it from every model except JevAny-Gemma-4B; the next six models are not separated from one another. The ranking follows the families only loosely: decoders with scorer, pointer or letter readouts fill the top of the table, but the encoder Von is eighth and the decoder OpenThai-SystemOne seventeenth.

Table 5 | JevBench-mini results. Overall coherence (the mean of the five dimension scores) with its 95% interval; the rank range lists the ranks the intervals allow, so models whose ranges overlap are not separated. Acc.: accuracy of the base answers against the gold answers (%). Below the line, the four reference models.

Rank	Range	Model	Overall	95% CI	REP	BAT	MEA	LOG	CHO	Acc.
1	1–2	Open-Jev-9B	83.9	82.5–85.2	81.9	100.0	50.2	87.1	100.0	88.4
2	1–7	JevAny-Gemma-4B (pointer)	81.2	79.0–83.2	81.4	100.0	52.0	83.8	88.9	87.6
3	2–7	JevAny-Qwen-4B (pointer)	79.4	77.2–81.4	81.7	100.0	52.2	78.3	84.7	88.8
4	2–8	decider-4b v2.1	78.4	76.0–80.7	80.3	100.0	52.5	78.8	80.6	88.6
5	2–8	JevAny-Qwen-4B (direct token)	77.8	75.4–80.2	81.4	100.0	50.2	80.8	76.4	88.6
6	2–8	TinyJev-4B	77.3	75.4–79.2	78.3	100.0	42.2	75.8	90.3	86.6
7	2–11	Jebadiah 9B v2	76.9	74.4–79.3	79.2	100.0	48.0	83.8	73.6	89.1
8	4–10	Von 1.3	75.9	74.7–77.1	83.1	100.0	16.4	80.0	100.0	72.2
9	7–15	JevK5 v0.3	72.8	70.4–75.3	76.1	100.0	43.6	76.2	68.1	88.1
10	7–15	OneJev-4B	72.5	69.9–74.7	79.7	100.0	43.6	71.2	68.1	89.2
11	8–15	RSI-Jev v3.0 2B	72.2	70.1–74.4	73.1	100.0	32.1	73.8	81.9	85.2
12	9–15	Kev-4B	71.9	69.8–73.9	74.4	100.0	30.1	67.5	87.5	87.9
13	9–15	decider-2b v10	71.7	69.2–74.4	78.6	100.0	38.0	67.1	75.0	84.9
14	9–15	jevhome-E	69.6	68.1–71.3	69.2	100.0	25.2	84.2	69.4	55.8
15	9–17	Bespoke-Nimble-9B ¹	69.2	66.5–72.1	76.4	68.3	50.5	87.1	63.9	89.1
16	15–19	CLM v0.1	66.8	65.5–68.1	59.2	100.0	25.5	49.6	100.0	63.5
17	15–20	OpenThai-SystemOne	64.2	61.2–67.0	76.9	70.8	39.7	61.3	72.2	82.4
18	16–19	jevhome-ettin-1b	64.1	61.8–66.2	70.3	100.0	29.2	70.8	50.0	82.3
19	16–20	Laya (typed-decisions)	63.9	61.4–66.3	55.0	100.0	14.5	73.8	76.4	71.0
20	18–21	Verdict 1.4 ²	59.4	57.0–61.8	66.1	100.0	19.9	77.9	33.3	64.2
21	20–21	Laya (english)	56.5	53.8–59.2	52.5	100.0	16.4	67.9	45.8	70.8
<i>Uniform (ignores the input)</i>			89.4	89.4–89.4	100.0	100.0	47.1	100.0	100.0	39.8
<i>Random (hash-seeded)</i>			34.5	33.5–35.5	18.6	100.0	4.9	48.8	0.0	39.6
<i>Luce toy scorer</i>			64.9	63.8–65.9	58.6	100.0	8.6	57.1	100.0	49.9
<i>Luce toy scorer, biased</i>			58.6	55.8–61.6	53.3	78.3	18.1	79.2	63.9	41.4

¹ Bespoke-Nimble-9B: JevBench shim (Nimble’s own Modal server does not run locally); same request mapping as Nimble’s compiler.py, PyTorch reference runner, current checkpoint. ² Verdict 1.4: OpenJev server (razorback16/openjev), which reimplements Verdict 1.4’s prompt and temperatures; Verdict ships no HTTP server.

Dimensions. Figure 1 breaks the scores into the eleven groups. Batch independence (BAT) is nearly saturated: 19 of the 21 models keep every batch law, and the two that do not are examined in §5.4. Representation (REP, mean 74.0), logical order (LOG, 75.1) and choice-set conditioning (CHO, 75.5) separate the models. Probability-measure coherence (MEA) is the lowest dimension of every model, with a mean of 36.8 and no model above 52.5.

Where models fail. The five hardest relations, by mean score over the models, are all in MEA: score_cumulative (1), choice_complement (6), score_complement (6), negation_wrapper (7), choice_indicator (8). They are the relations that ask the same event in two answer types or in negated form. A templated negation of a Noul (for example “Is it false that the answer to the following question is yes?”) sums to one with its source in 7% of tests on average, a natural negation from the rewrite bank in 42%. In 87% of the 468 violated templated tests, the model gives the question and its negation the same verdict (both above or both below one half), that is, it largely ignores the negation, as earlier studies found for masked language models [14]; for natural negations the share is 51%. The cumulative Nouls of a Score (“level k or higher”) almost never match the Score’s own tail probabilities (1%), and the Noul “is it o ?” rarely matches the probability of option o in the Choice (8%). These relations assume that two answer types encode the same event, and models that fail the answer-type group of REP tend to fail them too (§3.8); but models that keep the answer-type laws fail them as well: JevAny-Qwen-4B (direct token) keeps every answer-type test and scores 38.5 on marginalisation. No relation is kept by every model.

In LOG, the monotonicity of thresholds and quantifiers holds more often (group mean 79) than the bounds of conjunctions and disjunctions (73), and the spread is large: CLM keeps monotonicity in 21% of the tests,

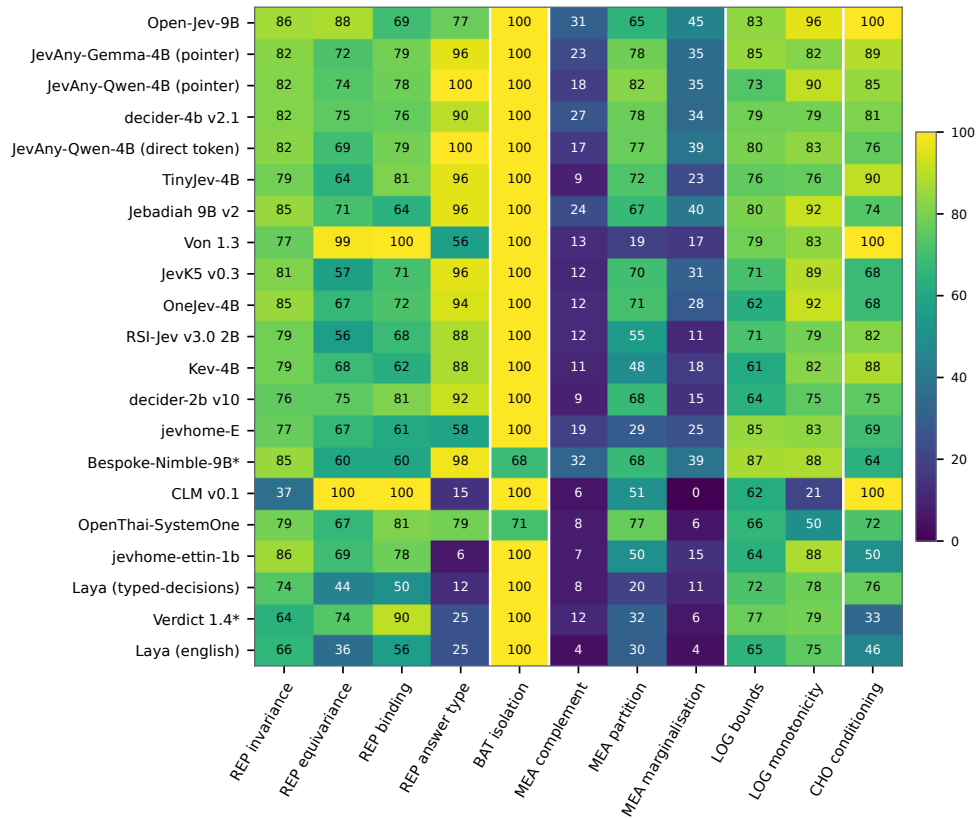


Figure 1 | Group scores on JevBench-mini, one row per model, ordered by overall coherence. White lines separate the five dimensions. *: not served by its developers' own server.

Open-Jev-9B in 96%. In CHO, three models keep independence of irrelevant alternatives in every test (Von, CLM and Open-Jev-9B), while Verdict keeps it in a third of the tests.

5.3 Coherence and accuracy

Coherence is necessary for a trustworthy decision model, not sufficient (§3.2). The reference models make this concrete (Table 5, Figure 2).

Why a model that ignores its input scores 89.4. The uniform model gives every answer of a question the same probability: one half for a Noul, one quarter for each option of a four-option Choice or level of a four-level Score. Its answers are not constant, since they depend on the number of answers, but they never depend on what the answers mean. The laws of JevBench are of two kinds, and the uniform model meets them very differently.

- *Laws that ask an answer to stay the same, to keep an order, or to condition on a menu.* Every REP law compares two descriptions of the same events, every BAT law the same question with different companions, every LOG law asks probabilities to respect an order, and every CHO law asks the distribution over the remaining options to be the renormalised original. A uniform answer keeps all of them: two descriptions of a four-option Choice both get one quarter per option, all propositions of a chain get one half (an order holds with equality), and a uniform distribution renormalised to any sub-menu is the uniform distribution of that sub-menu. REP, BAT, LOG and CHO all score 100.
- *Laws that ask probability to be split according to what the answers mean.* Most MEA laws compare different events of one measure: merging two options must add their probabilities, splitting one must divide it, a question about a single option must match that option's share. These laws fix how much the answer must move when the question changes, and the right amount depends on which events the options denote. The uniform model moves by an amount set only by the number of options, and fails every such law whose answer differs from a uniform split (Table 6).

The other eight MEA relations hold for a uniform answer on the tests of JevBench-mini: a Noul and its

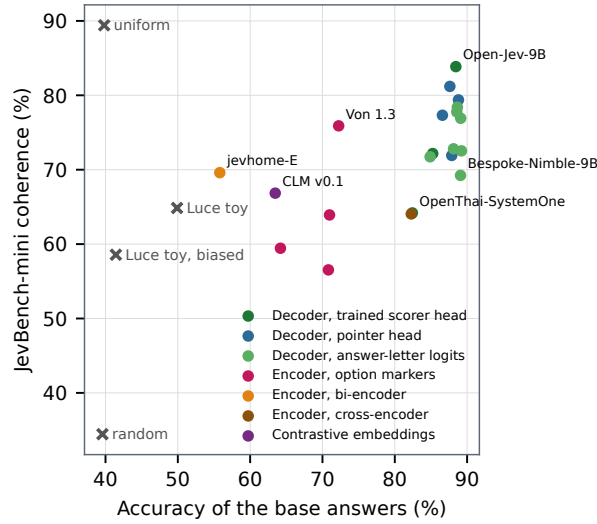


Figure 2 | Coherence on JevBench-mini against the accuracy of the base answers. Crosses: reference models. The uniform model keeps almost every law at chance accuracy; among decoders, accuracy varies little and coherence a great deal.

Table 6 | The nine relations the uniform model fails, on a four-option Choice or a four-level Score, the typical questions of the benchmark. Each law fixes a value from the source answer; the uniform model answers from the number of options alone. δ : violation measure (§3.3), against the tolerance 0.05.

Relation	What the law requires	Uniform answer	δ
option_merge	merged option $a \cup b$: $\frac{1}{4} + \frac{1}{4} = \frac{1}{2}$, others $\frac{1}{4}$	three options, $\frac{1}{3}$ each	0.17
option_split	the two parts of o sum to $P_q(o) = \frac{1}{4}$	five options, parts sum to $\frac{2}{5}$	0.15
option_clone	o and its copy sum to $\frac{1}{4}$	five options, sum $\frac{2}{5}$	0.15
similarity_effect	b and its near-duplicate sum to $\frac{1}{4}$	five options, sum $\frac{2}{5}$	0.15
extreme_level	new top level and old top sum to $\frac{1}{4}$	five levels, sum $\frac{2}{5}$	0.15
choice_indicator	“is it o ?” = $P_q(o) = \frac{1}{4}$	$\frac{1}{2}$	0.25
choice_complement	“not o ?” = $1 - P_q(o) = \frac{3}{4}$	$\frac{1}{2}$	0.25
score_complement	“not level k ?” = $1 - P_q(k) = \frac{3}{4}$	$\frac{1}{2}$	0.25
score_cumulative	“level k or higher?” = $\frac{3}{4}, \frac{1}{2}, \frac{1}{4}$	$\frac{1}{2}$ for every k	0.25

negation, templated, natural or of a conjunction, give $\frac{1}{2} + \frac{1}{2} = 1$; inclusion-exclusion gives $\frac{1}{2} + \frac{1}{2} = \frac{1}{2} + \frac{1}{2}$; a uniform four-way Choice over the answers to two Nouns has marginals $\frac{1}{2}$, equal to the Nouns; replacing an option by a catch-all keeps four options at one quarter each; and the coarsenings of these tests merge four outcomes into two blocks of two, so that the merged answer, one half per block, is again uniform. MEA is therefore $8/17 = 47.1$, and the overall score, the mean of the five dimensions, is $(4 \times 100 + 47.1)/5 = 89.4$.

The high score follows from the definition of coherence, not from a flaw in the scoring: answering without regard to the input is consistent with every law that does not depend on what the answers mean. We keep the five dimensions equally weighted, because each tests a different operation that a decision system applies to the answers, and we do not fold informativeness into the coherence score; instead we report accuracy beside it.

The accuracy of the reference models. Accuracy is computed from the base answers alone (§5.1). For the uniform model every answer of a question ties for the highest probability, so each question earns $1/k$ for its k answers: one half for each of the 1,152 Nouns with a known gold answer, one quarter for each of the 548 four-option Choices and 222 four-level Scores, and one fifth for each of the 16 five-level Scores. The mean over the 1938 questions is 39.8%, the accuracy of guessing. The random model, whose answers to different questions are unrelated, scores 34.5 at 39.6% accuracy; only BAT, which compares identical questions, holds in full. The two toy scorers fall between (64.9 and 58.6, at 49.9% and 41.4%).

Table 7 | Batch-independence relations (scores in %) for the two models that fail them, and the lowest score of the other models.

Model	batch solo	batch order	duplicate question	batch content	batch size sweep
Bespoke-Nimble-9B	87.5	4.2	100.0	91.7	58.3
OpenThai-SystemOne	83.3	12.5	66.7	91.7	100.0
Other 19 models (lowest)	100.0	100.0	100.0	100.0	100.0

Coherence and accuracy of the models. Across the 21 models, accuracy ranges from 55.8% to 89.2%, and its rank correlation with coherence is 0.64. The correlation comes mostly from the families: the option-marker encoders, the bi-encoder and CLM are the least accurate (55.8–72.2%) and mostly less coherent. Within the 14 decoders, accuracy lies in a narrow band (82.4–89.2%) while coherence spans 64.2–83.9. OneJev-4B, Jebadiah 9B and Bespoke-Nimble-9B are the three most accurate models (89.1–89.2%) and are ranked tenth, seventh and fifteenth on coherence. Some models reverse the order of the two measures: jevhome-E, a 48M bi-encoder, is more coherent than the 1B cross-encoder jevhome-ettin-1b distilled from the same teacher (69.6 against 64.1) and far less accurate (55.8% against 82.3%). The two measures thus answer different questions, and a model should be read on both: coherence without accuracy can be achieved by ignoring the input, and accuracy without coherence leaves the answers inconsistent once they are combined.

5.4 Batch independence

A question’s answer must not depend on the other questions of its request. Nineteen models keep this exactly, which is what serving each question independently guarantees. Two models fail it, in different ways (Table 7), and both failures follow from how the model reads a request.

OpenThai-SystemOne concatenates the state and all questions into one sequence and reads each answer at the answer token of its question. The backbone is causal, so each answer depends on the questions *before* it. Reversing the batch changes most answers (batch_order holds in 13% of tests), and a question asked twice in one request is often answered differently the second time (duplicate_question, 67%), because its second copy sees the first. Adding or removing questions at the end leaves the earlier answers unchanged (batch_size_sweep, 100%).

Bespoke-Nimble-9B shows the whole schema of the request, every question with its options, in the prompt of each question, and scores the requested one. Each answer therefore depends on *all* the other questions, their order and their number: batch_order holds in 4% of tests and batch_size_sweep in 58%. A question asked twice is answered the same both times, since the two prompts differ only in the name of the requested field. We checked that our adapter builds the same schema as the developers’ own request compiler, so this is a property of the model, not of the serving.

Both designs have a reason: reading all questions in one pass is cheaper, and seeing the other questions can help a model resolve what a question means. But they make an answer depend on how a client batches its questions, which a decision system rarely controls, and the effects can be large: for Bespoke-Nimble-9B, reordering moves an answer by more than 0.9 in total variation in the worst test.

5.5 Comparisons between related releases

Some releases are closely related and differ mainly in one factor. The comparisons are suggestive rather than controlled: related releases may also differ in training details, size or serving.

- *Readout and backbone, same data.* The three JevAny models were trained on the same 1.77M records. With a pointer readout, the Gemma backbone (81.2) and the Qwen backbone (79.4) are not separated; on the Qwen backbone, the pointer readout and the direct-token readout (77.8) are not separated overall; their largest difference is in CHO (84.7 against 76.4).
- *Task fine-tuning.* Laya typed-decisions is Laya fine-tuned on four workflows. Coherence rises from 56.5 to 63.9 at similar accuracy (70.8% and 71.0%).
- *Scale.* decider-4b is more coherent than decider-2b (78.4 against 71.7) and more accurate (88.6% against 84.9%). Across all models, the rank correlation between parameter count and coherence is 0.48.
- *Bi-encoder or cross-encoder.* The jevhome pair is discussed in §5.3: in this pair, the bi-encoder has the higher coherence and the cross-encoder the higher accuracy; the two also differ in size (48M and 1B).

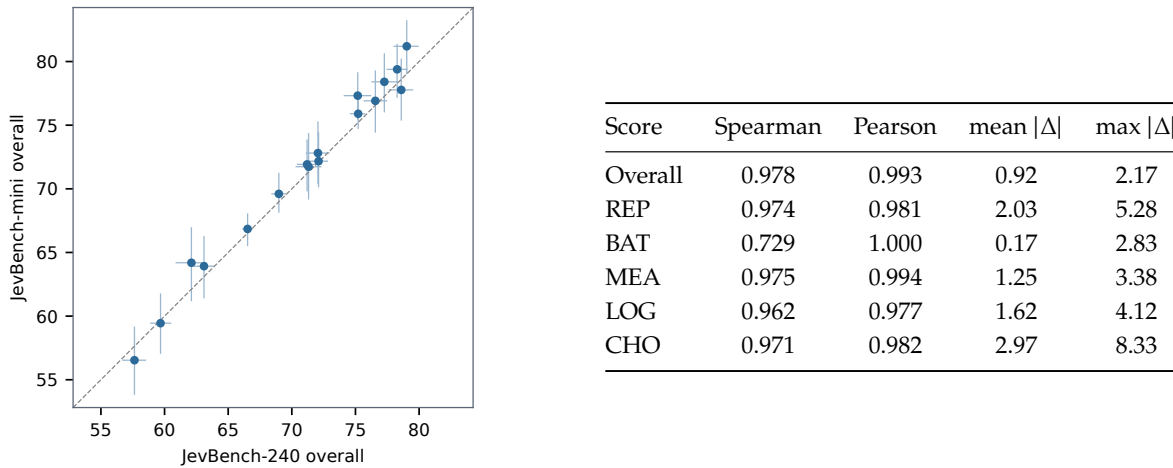


Figure 3 | Overall coherence of the 17 models run on both suites, with 95% intervals (left), and agreement of the overall and dimension scores (right; Δ : JevBench-mini minus JevBench-240, in points).

5.6 JevBench-mini against JevBench-240

JevBench-mini sends a ninth of the requests of JevBench-240. On the 17 models run on both (Figure 3), the overall scores agree closely: Pearson 0.993, Spearman 0.978, Kendall 0.897. JevBench-mini is higher by +0.66 points on average, at most 2.17 points apart, and no model moves by more than 2 ranks. Of the 136 pairs of models, 129 are ordered the same way, and all 118 pairs that JevBench-240 separates (95% intervals apart) are ordered the same way by JevBench-mini. The dimension scores agree as well, except that the rank correlation of BAT is lower (0.73) only because most models tie at 100. The price of the smaller suite is its intervals, about 2.5 times as wide (4.4 against 1.7 points on average): differences of a few points between models should be read as ties on JevBench-mini, and JevBench-240 separates them.

We do not compare models by domain. JevBench-mini tests each relation on two cases of a domain, fewer than the five a relation needs to be scored there. JevBench-240 has 20 cases per domain, but on the 17 models run on it a domain score differs from the model’s overall score by only 1.3 points on average, while its 95% interval (resampling the domain’s cases) spans ± 2.8 points; the overall score falls outside that interval for 8% of the (model, domain) pairs, about as often as chance alone would place it there. The differences between a model’s domains are within sampling noise.

5.7 Sensitivity to the tolerance

A test passes when its violation is at most $\varepsilon = 0.05$ in probability. Since every test’s violation is stored, the scores at any other tolerance follow from the same reports without a further request (Figure 4, Table 11 in Appendix D). Between $\varepsilon = 0.03$ and 0.1 the ranking of the 21 models barely changes: its rank correlation with the ranking at 0.05 is at least 0.95, no model moves by more than 4 ranks, and the first model stays first. Averaging each model’s score over all the tolerances we tried, from 0.001 to 0.2, gives a ranking with rank correlation 0.98 to the reported one, so the ranking does not rest on a lucky choice of ε .

At the extremes, the scores measure something else. At 0.001 (mean 41.4), a test passes only if the law holds to the third decimal, which rewards architectures that keep some laws exactly by construction: CLM, which scores each option independently, rises by 14 ranks. At 0.2 (mean 87.2), violations up to a fifth of the probability pass; the scores crowd together and Verdict, two thirds of whose violations lie between 0.05 and 0.2, rises by 15 ranks (rank correlation with the reported ranking 0.60). The uniform reference scores 89.4 up to $\varepsilon = 0.1$, since the laws it breaks miss by 0.15 to 0.25 (Table 6), and stays above every model at every tolerance (95.3 at 0.2).

5.8 Cost

A JevBench-mini run sends 1,248 requests. On one replica, the fastest model answers in 0.037 seconds per request, under a minute for the whole suite, and the slowest in 6.11 seconds, about two hours (Table 12 in Appendix E). The cost is set by how a server answers a request: one forward pass per question or per option, a sequential CPU runtime, or several passes per question. JevBench-240 takes about eight times as long.

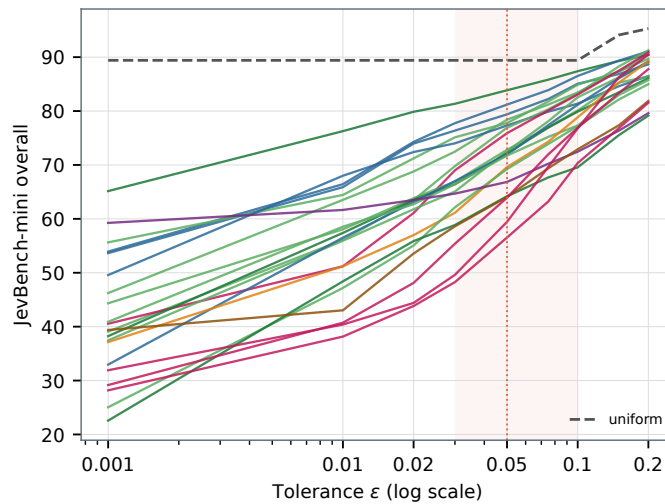


Figure 4 | Overall coherence on JevBench-mini as the tolerance varies, one line per model, coloured by family; dashed, the uniform reference. Every score rises with the tolerance (Proposition 2); the shaded band, $0.03 \leq \varepsilon \leq 0.1$, is where the ranking barely changes. Dotted, the reported tolerance.

Requests are cached, so a rerun after a change of the scoring sends nothing.

6 Discussion

What a coherence score means. A relation score is the share of tests in which a model’s answers keep one law within a tolerance of 0.05 in probability. It says nothing about whether the answers are right, and a model can raise it by answering less: the uniform model keeps every law that does not require a particular split of probability. We therefore report coherence beside accuracy and read a model on both (§5.3). The two are complementary in practice as well as in principle: among the decoder models, which dominate the field, accuracy on our cases falls within a relatively narrow range (82.4–89.2%), while coherence differs by twenty points.

Why the additive laws fail. Every model keeps fewer probability-measure laws than any other kind. These are the laws a downstream system relies on most directly: it negates conditions, merges options and compares a yes/no question with the Choice it summarises. Our results suggest that current training objectives, which fit each question’s distribution to a label, do not by themselves make the answers to related questions add up. Training on transformed questions with consistency targets, as has been proposed for question answering [15], is a natural direction; the frozen relations of JevBench could serve as such targets, provided the benchmark’s own cases are kept out of training.

Batching is part of the model. Two of the models make each answer depend on the other questions of a request. The interface allows a client to send any batch, so a client cannot know that its batching changes the answers unless the model documents it. JevBench makes the property measurable, and the two failure modes we found (dependence on earlier questions only, or on all of them) can be told apart by which batch relations fail.

7 Limitations

- *Synthetic, English data.* The cases are generated by one writer model and checked by one verifier, in English, in twelve business domains. They are balanced and their facts are verified, but they are not real traffic. The rewrite bank has been reviewed by models, and a sample of it by a model and by us. The gold answers used for accuracy are as reliable as the verifier and the answer rules; a spot check found natural vagueness in some of them, above all in Scores (§4.1), which affects the accuracy and no coherence score.
- *One fixed tolerance.* All laws are judged at 0.05 in probability. The ranking is stable from 0.03 to 0.1 and changes at the extremes, where the scores reward exact symmetries or forgive large violations (§5.7);

the package rescores saved results at any tolerance, and reports a graded score that measures how far beyond the tolerance violations go.

- *One test per relation and case.* A relation with several variants is tested on one of them per case, in turn. Its score estimates the share of (case, variant) pairs at which the law holds, which is never lower than the share of cases at which every variant holds (§3.3).
- *The reported subset.* JevBench-mini agrees with JevBench-240 on the ranking of the models we ran on both, but its intervals are about 2.5 times as wide. Neither suite separates a model’s domains: on JevBench-240 their differences are within sampling noise (§5.6).
- *Model coverage.* We evaluated openly released models built for the Jev interface. General-purpose instruction-tuned language models are not among them; any of them can be tested through the same interface, for example by reading the probabilities of the answer letters, and doing so is the natural next step.
- *Serving.* We served each model with its developers’ code at a pinned revision. Two models needed a third-party server or our adapter, and every model’s results reflect its released serving configuration (precision, prompt format, temperatures), not the best configuration it could have.
- *Repeatable answers.* No self-hosted model varied across three repetitions of the same request under the tested serving configurations, so a test compares one answer with one answer. A model that samples would add its own noise to every test; the package can measure that noise by repeating requests, but the benchmark judges such a model at the same tolerance.
- *A public test set.* The cases and tests are public. A model trained on them would no longer be measured fairly; the data generator is part of the package, so new cases can be generated for a private evaluation.

8 Ethical considerations

The cases are synthetic and describe fictional people and organisations; they contain no personal data. The evaluated models were used under their licences, for research. Coherence measures whether a model’s answers are consistent with one another, not whether they are correct or fair, and a high score should not be read as fitness for a decision task: a model that ignores its input scores higher than every model we evaluated (§5.3). Because the test set is public, scores of models trained on it would not be comparable (§7).

9 Conclusion

JevBench measures whether a typed probabilistic decision model answers related questions coherently, using 50 laws of probability and choice, without gold labels and with the same frozen test requests for every model. Across 21 openly released models, batch independence mostly holds, while additivity, above all for negations, fails broadly; and coherence and accuracy separate models in different ways, with a model that ignores its input as the clearest reminder that neither is enough alone. The benchmark, the data generator and the results are open source, and we hope they help make the answers of decision models fit together.

References

- [1] H. D. Block and Jacob Marschak. Random orderings and stochastic theories of responses. In Ingram Olkin et al., editors, *Contributions to Probability and Statistics*, pages 97–132. Stanford University Press, 1960.
- [2] Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.
- [3] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys*, 51(1):4:1–4:27, 2018.
- [4] Zhoujun Cheng, Jungo Kasai, and Tao Yu. Batch prompting: Efficient inference with large language model APIs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 792–810, 2023.

- [5] Bruno de Finetti. La prévision: ses lois logiques, ses sources subjectives. *Annales de l'Institut Henri Poincaré*, 7(1):1–68, 1937.
- [6] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.
- [7] Yanai Elazar, Nora Kassner, Shauli Ravfogel, Abhilasha Ravichander, Eduard Hovy, Hinrich Schütze, and Yoav Goldberg. Measuring and improving consistency in pretrained language models. *Transactions of the Association for Computational Linguistics*, 9:1012–1031, 2021.
- [8] Lukas Fluri, Daniel Paleka, and Florian Tramèr. Evaluating superhuman models with consistency checks. In *2024 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, pages 194–232, 2024.
- [9] Maurice Fréchet. Sur les tableaux de corrélation dont les marges sont données. *Annales de l'Université de Lyon, Section A*, 14:53–77, 1951.
- [10] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [11] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.
- [12] Joel Huber, John W. Payne, and Christopher Puto. Adding asymmetrically dominated alternatives: Violations of regularity and the similarity hypothesis. *Journal of Consumer Research*, 9(1):90–98, 1982.
- [13] Myeongjun Jang, Deuk Sin Kwon, and Thomas Lukasiewicz. BECEL: Benchmark for consistency evaluation of language models. In *Proceedings of the 29th International Conference on Computational Linguistics (COLING)*, pages 3680–3696, 2022.
- [14] Nora Kassner and Hinrich Schütze. Negated and misprimed probes for pretrained language models: Birds can talk, but cannot fly. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7811–7818, 2020.
- [15] Tao Li, Vivek Gupta, Maitrey Mehta, and Vivek Srikumar. A logic-driven framework for consistency of neural models. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019.
- [16] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, et al. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023.
- [17] R. Duncan Luce. *Individual Choice Behavior: A Theoretical Analysis*. Wiley, New York, 1959.
- [18] OpenAI. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.
- [19] Daniel Paleka, Abhimanyu Pallavi Sudhir, A. Alvarez, Vineeth Bhat, Adam Shen, Evan Wang, and Florian Tramèr. Consistency checks for language model forecasters. *arXiv preprint arXiv:2412.18544*, 2024.
- [20] Pouya Pezeshkpour and Estevam Hruschka. Large language models sensitivity to the order of options in multiple-choice questions. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 2006–2017, 2024.
- [21] Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4902–4912, 2020.
- [22] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models' sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. In *International Conference on Learning Representations (ICLR)*, 2024.
- [23] Sergio Segura, Gordon Fraser, Ana B. Sánchez, and Antonio Ruiz-Cortés. A survey on metamorphic testing. *IEEE Transactions on Software Engineering*, 42(9):805–824, 2016.
- [24] Itamar Simonson. Choice based on reasons: The case of attraction and compromise effects. *Journal of Consumer Research*, 16(2):158–174, 1989.

- [25] Amos Tversky. Elimination by aspects: A theory of choice. *Psychological Review*, 79(4):281–299, 1972.
- [26] Amos Tversky and Daniel Kahneman. Extensional versus intuitive reasoning: The conjunction fallacy in probability judgment. *Psychological Review*, 90(4):293–315, 1983.
- [27] Xiaoyuan Xie, Joshua W. K. Ho, Christian Murphy, Gail Kaiser, Baowen Xu, and Tsong Yueh Chen. Testing and validating machine learning classifiers by metamorphic testing. *Journal of Systems and Software*, 84:544–558, 2011.
- [28] Chujie Zheng, Hao Zhou, Fandong Meng, Jie Zhou, and Minlie Huang. Large language models are not robust multiple choice selectors. In *International Conference on Learning Representations (ICLR)*, 2024.

A Laws considered and not included

The relations of Table 3 are irredundant (§3.8). This appendix lists the further laws we considered, grouped by the reason each is not a relation: it follows from the laws of other relations on the same outputs (§A.1), it coincides with another relation on some tests (§A.2), or it does not satisfy Definition 1 (§A.3). Notation follows §3: P_q is the source output, $P_{q'}$ the follow-up output, p the probability of yes of a Noul, C the original options and $P|_K$ a distribution renormalised to K .

For each implied law we give the derivation at $\varepsilon = 0$ and a bound on its violation measure δ_w in terms of the measure δ_s of the relation that implies it. When $\delta_w \leq \delta_s$, every test that passes the stronger relation at tolerance ε passes the weaker law at the same tolerance; when the bound carries a factor, the implication holds exactly but may fail at a positive tolerance. We repeatedly use that a pointwise difference never exceeds the total variation, $|P(y) - Q(y)| \leq \text{TV}(P, Q)$, and that for $f : \mathcal{Y} \rightarrow [0, 1]$, $|\mathbb{E}_P f - \mathbb{E}_Q f| \leq \text{TV}(P, Q)$.

A.1 Laws implied by other relations

Regularity after an addition. When a menu grows by an alternative z , no original alternative should gain probability: $P_{q'}(a) \leq P_q(a)$ for every $a \in C$. Independence of irrelevant alternatives, $P_{q'}|_C = P_q$, gives

$$P_{q'}(a) = (1 - P_{q'}(z))P_{q'}|_C(a) = (1 - P_{q'}(z))P_q(a) \leq P_q(a), \quad (29)$$

and in general $P_{q'}(a) - P_q(a) \leq P_{q'}|_C(a) - P_q(a) \leq \text{TV}(P_{q'}|_C, P_q)$, so $\delta_w \leq \delta_s$ for `irrelevant_option`. When the addition refines an option b , as a near-duplicate b' (`similarity_effect`) or a level beyond the top of a Score (`extreme_level`), the partition law keeps every other outcome and gives $P_{q'}(b) + P_{q'}(b') = P_q(b)$, hence $P_{q'}(b) \leq P_q(b)$; each coordinate of $\phi_{\#}P_{q'} - P_q$ is at most δ_s , so again $\delta_w \leq \delta_s$.

Regularity after a removal. When an option o is removed, no remaining alternative should lose probability: $P_q(a) \leq P_{q'_o}(a)$. Independence, $P_{q'_o} = P_q|_{C \setminus \{o\}}$, gives $P_{q'_o}(a) = P_q(a)/(1 - P_q(o)) \geq P_q(a)$; since $P_q(a) \leq P_q|_{C \setminus \{o\}}(a)$, also $P_q(a) - P_{q'_o}(a) \leq P_q|_{C \setminus \{o\}}(a) - P_{q'_o}(a) \leq \delta_s$ for `remove_option_ia`.

Compromise effect. A level beyond the top of a Score should not draw probability towards the lower levels: $P_{q'}(k) \leq P_q(k)$ for every level k below the top. Under `extreme_level` every lower level keeps its probability, and $P_{q'}(k) - P_q(k) \leq \text{TV}(\phi_{\#}P_{q'}, P_q) = \delta_s$.

Exclusivity of option Nouns. The Nouns “is it o ?”, one per option, should sum to one. With $p_{q'_o} = P_q(o)$ for every option (`choice_indicator`), $\sum_o p_{q'_o} = \sum_o P_q(o) = 1$. In general

$$\left| \sum_o p_{q'_o} - 1 \right| = \left| \sum_o (p_{q'_o} - P_q(o)) \right| \leq k \delta_s \quad (30)$$

for k options: the implication is exact, but errors within tolerance can add up.

Monotonicity of cumulative Nouns. The Nouns “level k or higher” should not increase with k . Writing $\tau_k = \sum_{j \geq k} P_q(j)$, which decreases in k , `score_cumulative` requires $p_{q'_k} = \tau_k$, and for $k < k'$

$$p_{q'_{k'}} - p_{q'_k} = (p_{q'_{k'}} - \tau_{k'}) - (p_{q'_k} - \tau_k) + (\tau_{k'} - \tau_k) \leq 2 \delta_s. \quad (31)$$

Three of the four binary Fréchet bounds. Write $g = p_{A \wedge B} + p_{A \vee B} - p_A - p_B$ for the inclusion–exclusion gap (inclusion_exclusion). Then

$$\text{lower bound of } A \wedge B: p_A + p_B - 1 - p_{A \wedge B} = -g + (p_{A \vee B} - 1) \leq |g|, \quad (32)$$

$$\text{upper bound of } A \vee B: p_{A \vee B} - p_A - p_B = g - p_{A \wedge B} \leq |g|, \quad (33)$$

$$\text{lower bound of } A \vee B: \max\{p_A, p_B\} - p_{A \vee B} = (p_{A \wedge B} - \min\{p_A, p_B\}) - g. \quad (34)$$

The first two follow from inclusion–exclusion alone, since $p_{A \vee B} \leq 1$ and $p_{A \wedge B} \geq 0$. The third follows from inclusion–exclusion together with the upper bound of $A \wedge B$ (frechet_and_upper), with $\delta_w \leq \delta_{\text{frechet_and_upper}} + |g|$.

Fréchet bounds of the pair Choice. The four-way Choice over the answers to A and B gives a joint cell $P_{q'}(11)$, which should satisfy $p_A + p_B - 1 \leq P_{q'}(11) \leq \min\{p_A, p_B\}$. Once its marginals equal p_A and p_B (pair_choice_marginals), both bounds hold for any distribution over the four cells: $P_{q'}(11) \leq P_{q'}(1\cdot)$ and $P_{q'}(11) = P_{q'}(1\cdot) + P_{q'}(\cdot 1) - 1 + P_{q'}(00)$.

Restrictions of a relation’s own law. Three further checks restate part of a relation’s law. For a split option, the parts should sum to the whole, $P_{q'}(o \wedge X) + P_{q'}(o \wedge \neg X) = P_{q'}(o)$; this is one coordinate of option_split, $(\phi_{\#} P_{q'})(o) = P_{q'}(o)$. Proportionality of the renormalised option Nouns to the Choice follows from choice_indicator. For a near-duplicate b' of b , the requirement that every other option keep its probability is equivalent to similarity_effect, since the mass of b and b' together is then fixed by the others.

A.2 Coinciding tests

Two relations would share tests with another relation if their index were not restricted. A batch of size one is the question asked alone, so batch_size_sweep starts at size two and leaves that test to batch_solo. With three options, removing one leaves a pair, so pairwise_ia applies from four options on and leaves three-option menus to remove_option_ia.

A.3 Candidate laws outside the definition

Laws that need the correct answer. Three natural checks refer to the correct answer and so violate (P2): that the answers to “more than k ?” follow the step $\mathbb{I}[k < \text{true count}]$; that an off-topic option receives no probability, which presumes it is wrong; and that a “none of the above” option added to the full menu receives none. The last is not a law of the model’s outputs either: a Choice forces an answer among its options, so putting all mass on them does not assert that one of them holds.

Transformations chosen from the model’s answers. Removing the option the model finds least probable, or replacing the one it finds most probable by a catch-all, tests the same laws as remove_option_ia and option_closure. Because the removed option would depend on the model, different models would answer different questions, contrary to (P4). Removing or replacing every option in turn tests these laws on a fixed test set.

Hypothetical conditioning. The chain rule $p_{A \wedge B} = p_A p_{B|A}$ needs the answer to a conditional question, “supposing A , is B the case?”. When the state already settles A , such a supposition can contradict the state, and the conditional question has no answer that the law constrains.

Independence for near-duplicates. Independence of irrelevant alternatives is not a coherence law for an alternative that nearly duplicates an existing one: it would have the duplicate take share from every option in proportion, where the duplicate should take it from its original. This is the classical objection to the choice axiom for similar alternatives [25]. A near-duplicate refines its original, and JevBench tests it with the partition law (similarity_effect).

Decoys. A decoy is a weaker or partial version of a target option t . It overlaps t : an answer that fits the decoy also fits t , so the menu no longer lists disjoint events. Neither the conditioning law of CHO nor a partition law then applies, and the attraction effect has no coherence law on such a menu. A decoy disjoint from t would be an ordinary added option, subject to the conditioning law.

B Models

Table 8 lists the evaluated models by family, from their own documentation and server source. Each was installed at a pinned revision; the exact installation and serving commands are part of the benchmark’s repository. Table 9 groups the overall scores by readout family.

Table 8 | The 21 evaluated models.

Model	Developer	Base model	Readout and training	Params
<i>Decoder, trained scorer head</i>				
Open-Jev-9B	Zefan Cai	Qwen3.5-9B	rank-8 LoRA + scalar decision head; NLL + Brier	9.7B
OpenThai-SystemOne	iApp Technology	Qwen3.5-0.8B-Base (Thai continued pre-training)	256-slot decision head; questions read as one causal sequence	0.75B
RSI-Jev v3.0 2B	Shanghua Gao	Qwen3.5-2B-Base	trained readout/scorer head; SFT then RL, fitted calibration	2.3B
<i>Decoder, pointer head</i>				
JevAny-Gemma-4B (pointer)	SimpleJev	Gemma-4-E4B-it	LoRA SFT, pointer readout	E4B
JevAny-Qwen-4B (pointer)	SimpleJev	Qwen3.5-4B	LoRA SFT, pointer readout	4B
Kev-4B	Jared Palmer	Qwen3.5-4B-Base	rank-16 LoRA + pointer head	4.7B
TinyJev-4B	AnkitAI	Qwen3-4B-Base	merged LoRA + pointer head (Kev recipe)	4.0B
<i>Decoder, answer-letter logits</i>				
Bespoke-Nimble-9B ¹	Bespoke Labs	Qwen3.5-9B	LoRA scoring one-letter answer tokens; whole schema in every prompt	9.7B
Jebadiah 9B v2	Frontier Infra	Qwen3.5-9B	merged LoRA, per-type temperatures	9.65B
JevAny-Qwen-4B (direct token)	SimpleJev	Qwen3.5-4B	LoRA SFT, direct-token readout	4B
JevK5 v0.3	Alibi Serikbay	Qwen3.5-4B	distilled LoRA (merged), option-letter logit readout	4.2B
OneJev-4B	OmniJev	Qwen3.5-4B	full fine-tune (vision tower frozen); option-letter probability	5.2B
decider-2b v10	Mapika	Qwen3.5-2B-Base	fine-tune + calibration-aware RL; option-letter logits	1.9B
decider-4b v2.1	Mapika	Qwen3.5-4B-Base	full fine-tune + LoRA stage; option-letter logits, per-type temperatures	4.2B
<i>Encoder, option markers</i>				
Laya (english)	Convai Innovations	ModernBERT-large	one [MASK] marker per option	421M
Laya (typed-decisions)	Convai Innovations	ModernBERT-large	as Laya, fine-tuned on four workflows	421M
Verdict 1.4 ²	Heman10x	ModernBERT-base	GLiClass bi-encoder head, RLCD/Brier calibration	151M
Von 1.3	wfzyx	ModernBERT-large	option-marker head; listwise CE + Brier, temperature map	395M
<i>Encoder, bi-encoder</i>				
jevhome-E	Jibril Frej	granite-embedding-small-r2	bi-encoder distilled from a Qwen3.5-4B teacher	48M
<i>Encoder, cross-encoder</i>				
jevhome-ettin-1b	Jibril Frej	Ettin-encoder-1B	cross-encoder distilled from a Qwen3.5-4B teacher	1.0B
<i>Contrastive embeddings</i>				
CLM v0.1	Contrastive-LM	Qwen3-8B (frozen)	projection heads over frozen embeddings	8.2B + 20M

¹ Bespoke-Nimble-9B: JevBench shim (Nimble’s own Modal server does not run locally); same request mapping as Nimble’s compiler.py, PyTorch reference runner, current checkpoint. ² Verdict 1.4: OpenJev server (razorback16/openjev), which reimplements Verdict 1.4’s prompt and temperatures; Verdict ships no HTTP server.

Table 9 | Overall coherence on JevBench-mini by readout family: number of models, mean and range.

Readout family	Models	Mean	Range
Decoder, trained scorer head	3	73.4	64.2–83.9
Decoder, pointer head	4	77.5	71.9–81.2
Decoder, answer-letter logits	7	74.2	69.2–78.4
Encoder, option markers	4	63.9	56.5–75.9
Encoder, bi-encoder	1	69.6	69.6–69.6
Encoder, cross-encoder	1	64.1	64.1–64.1
Contrastive embeddings	1	66.8	66.8–66.8

C Scores by relation

Figure 5 shows every relation score of every model on JevBench-mini, and Table 10 summarises each relation over the models.

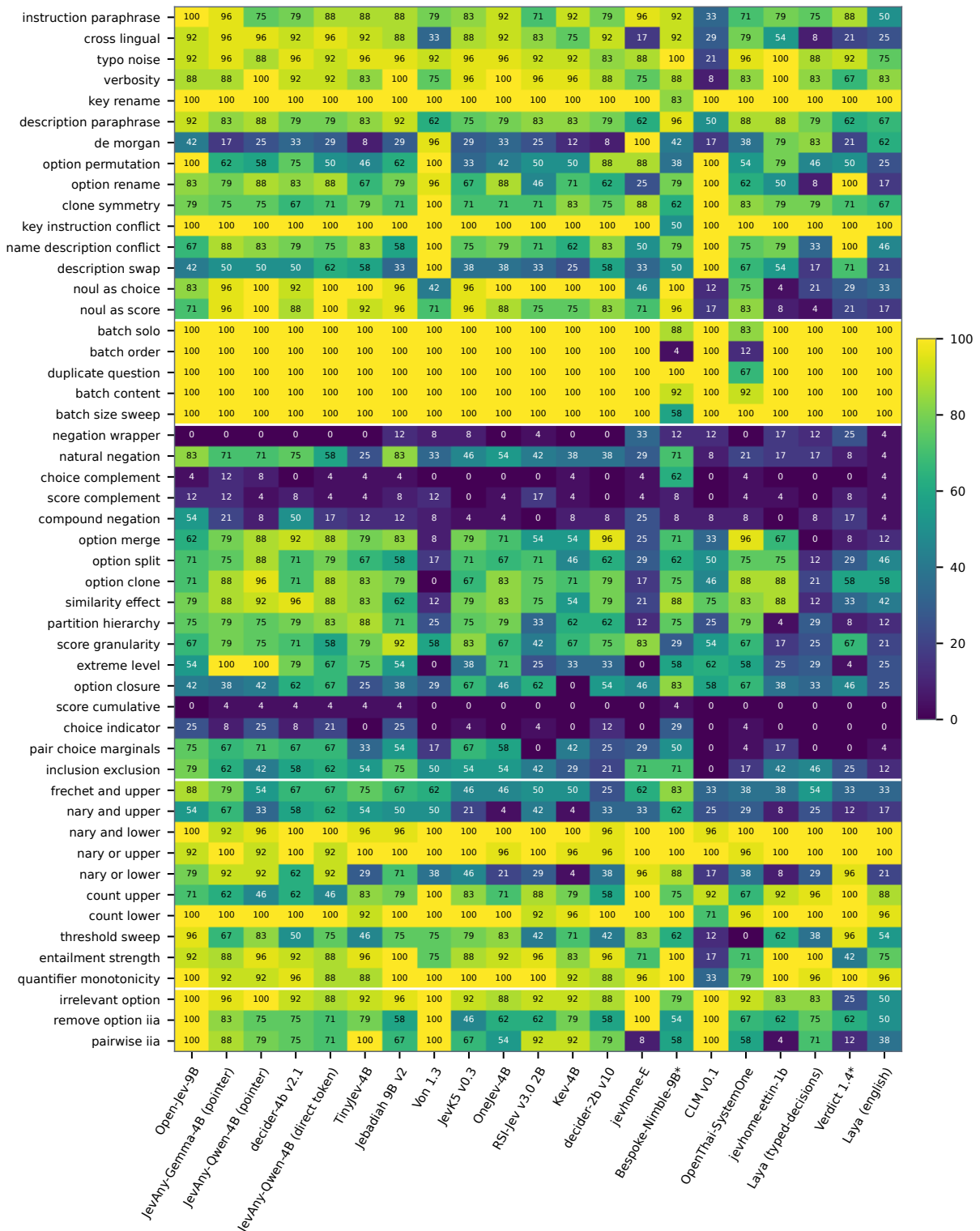


Figure 5 | Relation scores on JevBench-mini (%), one column per model, ordered by overall coherence. White lines separate the dimensions, in the order REP, BAT, MEA, LOG, CHO. *: not served by its developers' own server.

Table 10 | Relation scores on JevBench-mini across the 21 models: mean, lowest and highest, and the number of models at 100.

Group	Relation	Mean	Min	Max	# at 100
REP.invariance	instruction_paraphrase	80.6	33.3	100.0	1
	cross_lingual	68.5	8.3	95.8	0
	typo_noise	88.7	20.8	100.0	2
	verbosity	84.7	8.3	100.0	4
	key_rename	99.2	83.3	100.0	20
	description_paraphrase	78.6	50.0	95.8	0
	de_morgan	39.5	8.3	100.0	1
REP.equivariance	option_permutation	61.7	25.0	100.0	3
	option_rename	68.5	8.3	100.0	2
	clone_symmetry	77.0	62.5	100.0	2
REP.binding	key_instruction_conflict	97.6	50.0	100.0	20
	name_description_conflict	74.6	33.3	100.0	3
	description_swap	50.0	16.7	100.0	2
REP.type	noul_as_choice	72.6	4.2	100.0	8
	noul_as_score	68.8	4.2	100.0	2
BAT.isolation	batch_solo	98.6	83.3	100.0	19
	batch_order	91.3	4.2	100.0	19
	duplicate_question	98.4	66.7	100.0	20
	batch_content	99.2	91.7	100.0	19
	batch_size_sweep	98.0	58.3	100.0	20
MEA.complement	negation_wrapper	7.1	0.0	33.3	0
	natural_negation	42.5	4.2	83.3	0
	choice_complement	5.6	0.0	62.5	0
	score_complement	6.0	0.0	16.7	0
	compound_negation	13.7	0.0	54.2	0
MEA.partition	option_merge	59.3	0.0	95.8	0
	option_split	58.1	12.5	87.5	0
	option_clone	66.7	0.0	95.8	0
	similarity_effect	67.3	12.5	95.8	0
	partition_hierarchy	54.0	4.2	87.5	0
	score_granularity	60.7	16.7	91.7	0
	extreme_level	47.2	0.0	100.0	2
	option_closure	46.0	0.0	83.3	0
MEA.marginalisation	score_cumulative	1.4	0.0	4.2	0
	choice_indicator	7.9	0.0	29.2	0
	pair_choice_marginals	35.5	0.0	75.0	0
	inclusion_exclusion	46.0	0.0	79.2	0
LOG.bounds	frechet_and_upper	54.8	25.0	87.5	0
	nary_and_upper	35.5	4.2	66.7	0
	nary_and_lower	98.6	91.7	100.0	15
	nary_or_upper	98.0	91.7	100.0	14
	nary_or_lower	51.6	4.2	95.8	0
	count_upper	78.0	45.8	100.0	3
	count_lower	97.2	70.8	100.0	15
LOG.monotonicity	threshold_sweep	61.5	0.0	95.8	0
	entailment_strength	83.5	16.7	100.0	4
	quantifier_monotonicity	92.1	33.3	100.0	9
CHO.conditioning	irrelevant_option	86.9	25.0	100.0	5
	remove_option_ia	72.4	45.8	100.0	4
	pairwise_ia	67.3	4.2	100.0	4

D Scores at other tolerances

Table 11 | JevBench-mini at each tolerance: the rank correlations of the ranking with the one at $\varepsilon = 0.05$, the largest change of rank, the mean score of the 21 models, the uniform reference and the first model.

ε	Spearman	Kendall	max rank shift	mean score	uniform	first
0.001	0.738	0.600	14	41.4	89.4	Open-Jev-9B
0.01	0.858	0.705	9	55.7	89.4	Open-Jev-9B
0.02	0.926	0.810	6	62.0	89.4	Open-Jev-9B
0.03	0.978	0.905	3	66.0	89.4	Open-Jev-9B
0.05	1.000	1.000	0	71.7	89.4	Open-Jev-9B
0.075	0.973	0.886	3	76.2	89.4	Open-Jev-9B
0.1	0.953	0.838	4	79.6	89.4	Open-Jev-9B
0.15	0.778	0.608	12	84.2	94.1	JevAny-Gemma-4B (pointer)
0.2	0.599	0.448	15	87.2	95.3	Jebadiah 9B v2

E Runtime

Table 12 | Seconds per request for one server on one A100 40GB, measured on each model's first run, and the resulting time of a JevBench-mini run (1,248 requests) and of a JevBench-240 run (10,320 requests) on one replica. [†]: runs on CPU (ONNX Runtime, 12 threads).

Model	Params	s / request	mini, min	240, min
Verdict 1.4	151M	0.037	0.8	6
Laya (english)	421M	0.055	1.1	9
Laya (typed-decisions)	421M	0.056	1.2	9
CLM v0.1	8.2B + 20M	0.066	1.4	8
Kev-4B	4.7B	0.083	1.7	10
OpenThai-SystemOne	0.75B	0.134	2.8	22
RSI-Jev v3.0 2B	2.3B	0.155	3.2	26
TinyJev-4B	4.0B	0.175	3.6	29
decider-2b v10	1.9B	0.194	4.0	33
jevhome-E [†]	48M	0.232	4.8	40
JevK5 v0.3	4.2B	0.351	7.3	61
JevAny-Gemma-4B (pointer)	E4B	0.393	8.2	68
decider-4b v2.1	4.2B	0.416	8.7	71
Jebadiah 9B v2	9.65B	0.429	8.9	74
OneJev-4B	5.2B	0.463	9.6	–
JevAny-Qwen-4B (pointer)	4B	0.511	10.6	89
JevAny-Qwen-4B (direct token)	4B	0.530	11.0	91
Von 1.3	395M	1.188	24.7	195
Bespoke-Nimble-9B	9.7B	2.620	54.5	–
Open-Jev-9B	9.7B	4.147	86.2	–
jevhome-ettin-1b [†]	1.0B	6.109	127.1	–